UNIVERSITY
OF OSLO

Master's Thesis

ON NEUROMORPHIC COMPUTING

With Spiking Neural Networks

Brage Wiseth

Departement of Informatics
Faculty of Mathematics and
Natural Sciences

Philip Haflinger - Supervisor
Yngve Hafting - Co-Supervisor

2026-04-22

ABSTRACT

The development of modern Deep Learning has achieved unprecedented performance across various domains, yet it remains fundamentally bottlenecked by the energy and memory inefficiencies of the von Neumann architecture. To address these limitations, this thesis investigates Neuromorphic Computing with Spiking Neural Networks (SNNs) as a biologically plausible, highly energy-efficient alternative. By shifting from synchronous, continuous-value matrix multiplications to asynchronous, event-driven sparse computations, neuromorphic systems emulate the physical principles of the biological brain.

This work explores the implementation of these principles on standard CPU/GPU hardware. Two primary methodologies are developed and evaluated on visual classification tasks: (1) an inference-optimized SNN that translates weights from a conventionally trained Fully Connected Network (FCN) using Time-To-First-Spike (TTFS) temporal encoding, and (2) an unsupervised, biologically inspired learning simulator incorporating structural plasticity (dynamic synaptogenesis and pruning). The results demonstrate the viability of temporal coding and local learning rules in extracting meaningful features from visual stimuli, highlighting the potential of neuromorphic algorithms to drastically reduce the computational footprint of artificial intelligence.

Wordcount: 19965

ACKNOWLEDGEMENTS & DECLARATIONS

I would like to thank my supervisors and the very kind and helpful comunity at the ROBIN and NANO research groups at the Department of Informatics.

The repository containing all source code including simulation software, source code for this document and its figures can be found at

DECLARATION OF THE USE OF GENERATIVE ARTIFICIAL INTELLIGENCE

In this scientific work, generative Artificial Inteligence (AI) has been used. All data and personal information have been processed in accordance with the University of Oslo's regulations, and I, as the author of the document, take full responsibility for its content, claims, and references. An overview of the use of generative AI is provided below.

The service Gemini, developed by Google [1], has been used to improve the content of the thesis. Sections of text like a subsection, paragraph or source code for figures was given to the model along with prompts to make the language free of errors and more professional. The text underwent multiple iterations using refined prompts to ensure structural coherence and academic tone. In the case for figures the model was used to speed up the development of the figures with the model generating numbers for positioning elements. The final result was cut out, fact-checked, and partly rewritten by the author.

CONTENTS

GLOSSARY

action potential: A rapid change in voltage across a neuron's cell membrane that propagates down the axon to the synapses.

***AER* – Adress Event Representation**

***AI* – Artificial Inteligence**

***ALU* – Arithmetic Logic Unit**

***ANN* – Artificial Neural Network**

***CNN* – Convolutional Neural Network**

***DAG* – Directional Acyclic Graph**

***DL* – Deep Learning**

***FCN* – Fully Connected Network**

***FPGA* – Field Programmable Gate Array**

***GLIF* – Generalized Leaky Integrate and Fire**

***GPT* – Generative Pre-trained Transformer**

***IF* – Integrate and Fire**

***LIF* – Leaky Integrate and Fire**

***LLM* – Large Language Model**

***LSTM* – Long Short-Term Memory**

***LTD* – Long-Term Depression**

***LTP* – Long-Term Potentiation**

***MAC* – Multiply And Accumulate**

***MLP* – Multi Layer Perceptron**

***PSC* – Post Synaptic Current**

***RNN* – Recurrent Neural Network**

saccade: A rapid, simultaneous movement of both eyes that abruptly shifts the point of fixation from one area of interest to another in the visual field.

***SNN* – Spiking Neural Network**

spike: An action potential conceptualized as a discrete, instantaneous event. In neuromorphic and computational models, it represents a binary signal traveling through the network.

spike train: A sequence of action potentials (spikes) occurring over time. It typically represents the temporal firing pattern of a single neuron.

***STDP* – Spike-Timing-Dependent Plasticity**

***SyOPs* – Synaptic Operations**

***TTFS* – Time-To-First-Spike**

***VLSI* – Very Large Scale Integration**

***WTA* – Winner-Take-All**

1 • INTRODUCTION

The development of intelligent machines is a significant objective in modern science and engineering. While the concept has historical roots in philosophy and early automata, the field has transitioned from speculative theory to practical application. Currently, artificial intelligence is central to technological and economic development. Understanding the mechanisms of intelligence and reproducing them in synthetic systems offers the potential for improved analysis of biological minds and the creation of tools for applications ranging from personalized medicine to automated scientific discovery.

In recent years, great strides has been made towards this goal. Deep Learning, which utilizes multilayered neural networks, has exceeded previous performance benchmarks. These systems have demonstrated high proficiency in tasks previously limited to human capability. For example, AlphaFold has addressed complex problems in protein folding [2], reinforcement learning agents have mastered the complexity of games such as Go [3], and Large Language Models have shown capabilities in text generation that approach human fluency. Consequently, AI is increasingly viewed as a general-purpose technology that may influence societal infrastructure.

However, despite these advances, there are significant limitations to the current approach. The success of modern deep learning relies heavily on scaling, which involves increasing data volume and computational power. This strategy is approaching physical and economic boundaries. Training state-of-the-art models consumes substantial energy and results in a large carbon footprint [4]. Although specialized hardware allows for more efficient computations, the underlying architecture and algorithms imposes an intrinsic limit on scalability independent of the underlying hardware. Furthermore, the requirement for massive datasets presents challenges in sourcing and curation. Additionally, evidence suggests that this scaling approach yields diminishing returns. Models often function as statistical correlation engines; they lack common-sense reasoning, struggle with out-of-distribution generalization, and are prone to brittle failure modes [5].

These limitations are evident when comparing artificial systems to biological intelligence. The human brain demonstrates that high-level intelligence is possible without massive energy consumption or dataset sizes. The brain operates on approximately 20 watts [6]. With this limited energy budget, it manages biological functions, processes real-time multi-sensory data, and performs abstract reasoning. In contrast, deep learning models require GPU clusters with significantly higher power requirements to match a fraction of these capabilities. There is also a discrepancy in learning efficiency. Deep learning models are sample-inefficient, often requiring vast numbers of examples to learn a representation. Biological systems, however, are capable of “one-shot” or “few-shot” learning and can acquire new information

without catastrophic forgetting. This suggests the inefficiency of current AI is a paradigmatic issue rather than just an engineering problem.

The proposed direction for addressing these issues involves biological inspiration in both hardware and algorithm design, specifically Neuromorphic Computing. This field attempts to engineer computer architectures that mimic the biological structure of the nervous system. Unlike traditional AI, which runs as software on general-purpose hardware, neuromorphic engineering aims to align the algorithm with the physical substrate. It moves away from clock-driven processing toward asynchronous, event-driven systems. In this paradigm, information is encoded as sparse, discrete events or "spikes". Similar to biological neurons, a neuromorphic processor consumes minimal energy when inactive, processing information only when triggered. This approach is being pursued by both industrial groups, such as Intel's Loihi [7], and academic projects like SpiNNaker [8]. These systems represent a shift from calculation-based machines to those capable of real-time adaptation.

Although neuromorphic systems achieve optimal performance on co-designed platforms—where the algorithm is embedded directly into the hardware—there is significant value in executing neuromorphic algorithms on traditional von Neumann architectures. In this thesis, we explore biologically inspired algorithms deployed on traditional CPU and GPU hardware. We examine how event-driven, biologically plausible computation can address limitations in scalability, data efficiency, and energy consumption, even when simulated on standard processors. We present approaches for efficient information coding and learning algorithms inspired by neural mechanisms. Concretely, this thesis aims to:

Investigate Sparse Information Flow: Explore how information can be encoded and processed using sparse, asynchronous events (spikes) within a neural network.

Develop Biologically Inspired Learning: Explore and evaluate learning algorithms that are suitable for such networks, adhering to the constraints of locality and efficiency.

The succeeding chapter lays the historical and theoretical foundation, covering early neuroscience and the development of artificial neural networks based on simple models of the brain. Following this, we review relevant modern neuroscience literature, extracting key concepts that will inform the methodology. We also provide consise overview on machine learning concepts and frameworks. Finally, we detail the implementation of these principles in a neuromorphic context and evaluate their performance against standard benchmarks.

2 • HISTORY & DEVELOPMENTS

Historically, the understanding of neural tissue was dominated by the reticular theory, which claimed that the brain consisted of a continuous, fused network of nerve fibers. This paradigm was fundamentally challenged by the work of Santiago Ramón y Cajal. Through the application of novel staining techniques, Cajal established the neuron doctrine, demonstrating that the nervous system is composed of discrete, individual cells [9]. Building on these findings, Heinrich Wilhelm Gottfried Von Waldeyer-Hartz proposed the "Neuron Doctrine" and coined the term "neurons" to describe these dicrete cells [10]. Subsequent analysis using electron microscopy has provided irrefutable validation of this discrete cellular structure.

The conceptualization of the brain as a collection of discrete units facilitated the development of mathematical models describing neural function. In 1943, Warren McCulloch and Walter Pitts published A Logical Calculus of the Ideas Immanent in Nervous Activity, introducing the first formal model of the neuron.

The McCulloch-Pitts (M-P) neuron abstracted biological complexity into a binary decision device governed by the following logic:

Inputs: The neuron receives multiple binary inputs, weighted as either excitatory or inhibitory.

Summation: The unit calculates the linear sum of these weighted inputs.

Thresholding: If the aggregate sum exceeds a fixed threshold, the neuron outputs a 1 (firing); otherwise, it outputs a 0 (silence).

McCulloch and Pitts demonstrated that networks of these units could theoretically compute any logical operation (AND, OR, NOT) [11]. This abstraction established the foundational link between biological processes and digital logic, suggesting that neural function could be replicated in electronic hardware. Consequently, the M-P neuron serves as the common ancestor for both computational neuroscience and artificial intelligence.

However, despite its theoretical utility, the original M-P model presented significant functional limitations. The connectivity was static, requiring circuits to be manually designed rather than learned. Furthermore, the restriction to binary weights precluded the modeling of graded signal intensity, preventing the system from capturing the nuance of real-world sensory input.

In 1949, Donald Hebb addressed the critical issue of plasticity in his work The Organization of Behavior. He proposed a theoretical mechanism for synaptic modification, now known as Hebbian learning, which provided a biological basis for how neural networks could adapt over time. Hebb postulated:

> "Let us assume that the persistence or repetition of a reverberatory activity (or "trace") tends to induce lasting cellular changes that add to its stability. ... When an axon of cell A is near enough to excite a cell B and repeatedly or persistently takes part in firing it, some growth process or metabolic change takes place in one or both cells such that A's efficiency, as one of the cells firing B, is increased" [12].

This principle is colloquially summarized as "neurons that fire together, wire together" [13]. Crucially, this describes a local and decentralized learning rule; a synapse does not require a global error signal or external supervision to adjust. It requires only the correlation between the pre-synaptic input and the post-synaptic output. The convergence of the M-P architectural model and the Hebbian plasticity framework established the prerequisite conditions for the development of modern neural networks.

2.1 • THE PERCEPTRON

In 1957, Frank Rosenblatt advanced these theoretical concepts by engineering the Perceptron [14]. The "Mark I Perceptron" was a hardware implementation of the neural model, distinguished by a crucial innovation: a weight-adjustment mechanism based on Hebbian principles. Rosenblatt introduced the perceptron learning rule, an iterative algorithm capable of minimizing error automatically. The system processed an input pattern (e.g., a pixelated character) and produced a binary classification. When the output deviated from the target, the algorithm adjusted the weights proportional to the error: strengthening connections that should have contributed to a correct firing and weakening those that led to false positives.

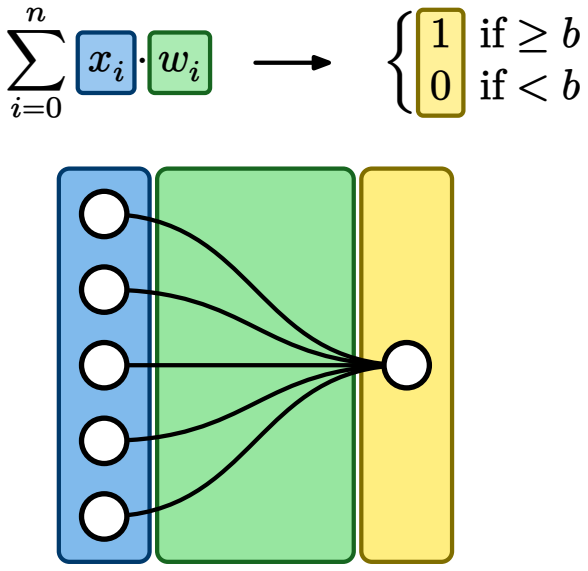

Figure 1: The perceptron model. Inputs x_i are multiplied by weights w_i and summed. If the linear combination $\sum x_i w_i$ exceeds the bias b, the neuron activates.

Consequently, the Perceptron was capable of converging on a solution for any problem where the data was linearly separable. This success generated significant enthusiasm, with contemporary reports suggesting that such machines would soon mimic human consciousness [15].

These expectations were abruptly tempered by theoretical limitations. In 1969, Marvin Minsky and Seymour Papert published Perceptrons, a rigorous mathematical analysis of the architecture [16]. They demonstrated that a single-layer perceptron is fundamentally a linear classifier. While capable of learning operations like AND or OR, it is mathematically incapable of solving the XOR (Exclusive OR) problem. In the XOR case, the classes cannot be separated by a single hyperplane. This proof highlighted a severe boundary on the utility of single-layer networks for complex, non-linear tasks.

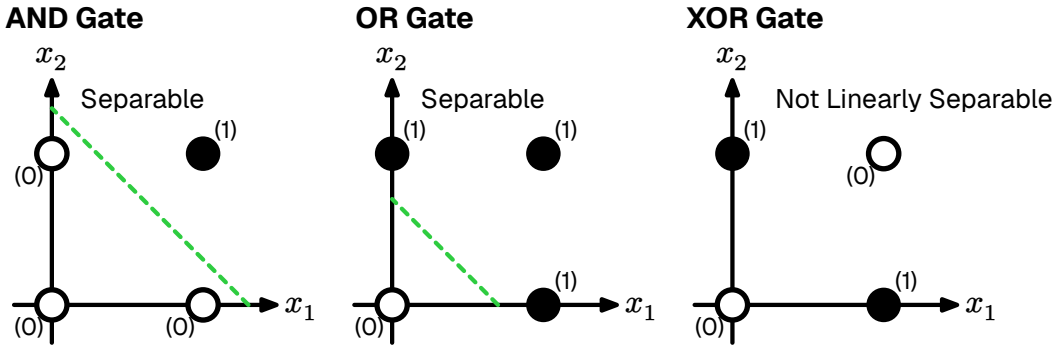

Figure 2: The XOR problem. Unlike AND/OR, the data points for XOR cannot be separated by a single linear boundary.

The publication of Perceptrons coincided with a significant reduction in neural network research funding, a period retrospectively termed the “First AI Winter”. It is worth noting that Minsky and Papert acknowledged that a Multi Layer Perceptron (MLP), a network stacking multiple layers of neurons, could theoretically solve the XOR problem by creating complex, non-linear decision boundaries.

However, a critical algorithmic gap remained: the “credit assignment problem”. While researchers knew that hidden layers could represent complex features, there was no known method to propagate error signals back through the layers to adjust the weights of hidden neurons effectively. Rosenblatt’s rule was mathematically valid only for the output layer. The field remained stagnant until a method for training multi-layer networks could be formalized.

2.2 • DEEP LEARNING

The critique presented by Minsky and Papert precipitated a contraction in funding; despite this, theoretical inquiry persisted. It was widely hypothesized that the limitations of the single perceptron could be overcome by a MLP. By organizing neurons (single perceptrons) into hierarchical layers, the network could theoreti-

cally perform successive non-linear transformations on the input space, enabling the formation of complex decision boundaries. The primary impediment was not the architecture itself, but the absence of a viable learning algorithm.

In a single-layer perceptron, error attribution is immediate: if the output deviates from the target, the error is directly derived from the weights of the output layer. However, in a multi-layer architecture, quantifying the contribution of a specific neuron within the "hidden" layers to the final output error presents a significant challenge. This is formally known as the Credit Assignment Problem [17], and it remained the central theoretical obstacle for over a decade.

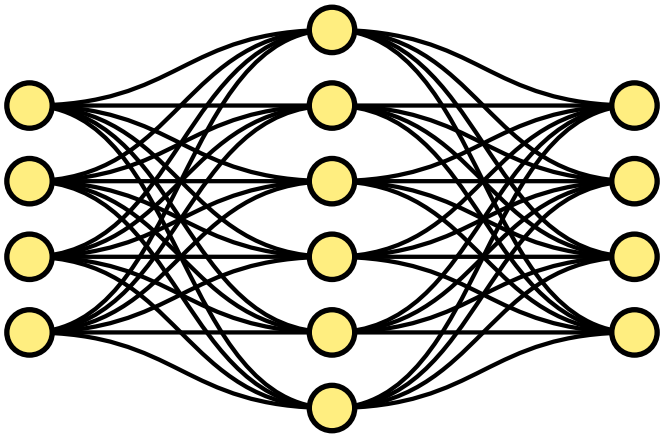

Figure 3: A Multi Layer Perceptron (MLP). By inserting "hidden layers" between input and output, the network can approximate non-linear functions such as XOR. The historical challenge lay in deriving a method to train these intermediate layers.

The solution to this theoretical impasse was popularized in 1986 by Rumelhart, Hinton, and Williams in their seminal paper *Learning representations by back-propagating errors* [18]. They demonstrated that the Chain Rule of calculus could be applied recursively to propagate the error signal from the output layer backwards through the hidden layers. This algorithm, known as Backpropagation, allowed the network to calculate the gradient of the loss function with respect to every weight in the system. Effectively, it provided a mathematical method to tell each hidden neuron exactly how much it contributed to the total error, finally solving the credit assignment problem.

Unlike Hebbian plasticity, which is local and biological, Backpropagation relies on global error signals and precise backward data flow—mechanisms effectively absent in organic tissue. Consequently, the field of Artificial Neural Network (ANN) effectively decoupled from neuroscience. It transitioned into a branch of engineering and applied mathematics, prioritizing statistical optimization over biological realism. Paradoxically, it was this abandonment of biological fidelity that enabled the rapid scaling and performance breakthroughs that followed.

2.2.1 • ACHIEVEMENTS

With the training mechanism solved, the field exploded. The combination of Backpropagation, massive datasets, and GPU hardware led to a led to a rapid diversification of neural architectures, each solving domains previously thought impossible for computers.

The revolution began in earnest with computer vision. Convolutional Neural Networks (CNNs), such as AlexNet (2012) [19] and later ResNet [20], introduced the idea of learning hierarchical features—detecting edges, then shapes, then objects—much like the human visual cortex. This allowed machines to classify images with superhuman accuracy.

Soon after, the focus shifted to sequence data. Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) architectures gave machines a short-term memory, enabling breakthroughs in speech recognition and machine translation. However, the true paradigm shift occurred with the introduction of the Transformer architecture in 2017 [21]. By utilizing an "attention mechanism" to parallelize the processing of language, Transformers allowed for the training of massive Large Language Models (LLMs) like the Generative Pre-trained Transformer (GPT).

These techniques have even transcended media generation. Deep Learning has solved fundamental scientific problems; notably, DeepMind's AlphaFold utilized these architectures to predict the 3D structure of proteins from their amino acid sequences, a 50-year-old grand challenge in biology [2]

2.2.2 • SHORTCOMINGS

Deep learning's reliance on computational scaling masks fundamental inefficiencies in both its hardware implementation and underlying algorithms. By simulating biological concepts on digital architectures not designed for them, the current paradigm is approaching physical and economic limits.

A primary limitation is the Von Neumann architecture, which physically separates processing units from memory. Deep neural networks, defined by massive matrices of synaptic weights, necessitate constant data transfer. For every inference step, billions of parameters must be fetched from off-chip DRAM, processed, and written back. This creates a severe memory bottleneck where system performance is bounded by bandwidth rather than processing speed [22].

Consequently, the energy cost of moving data significantly exceeds the cost of computation itself. Retrieving a single byte from DRAM consumes approximately three orders of magnitude more energy than performing a floating-point operation [23]. Compounding this hardware friction, the dense matrix multiplications required for training scale quadratically with network size, making the pursuit of trillion-parameter models increasingly unsustainable.

Furthermore, the optimization algorithms driving this scale are fundamentally incompatible with physical biological systems. Backpropagation, while mathematically elegant, relies on a global error signal and suffers from the "weight transport problem"—the requirement that the backward pass utilizes the exact same synaptic weights as the forward pass. In organic tissue, synapses are unidirectional, and there is no known mechanism for a neuron to access the exact weight of a downstream synapse to calculate a gradient.

While a detailed technical analysis of these inefficiencies is presented in Section 4, the central issue is clear: modern AI prioritizes statistical optimization over physical realism. Overcoming the limitations of the Von Neumann bottleneck and backpropagation requires a paradigm shift toward architectures that inherently co-locate memory and computation.

2.3 • BIRTH OF NEUROMORPHIC

While the artificial intelligence community debated symbolic logic versus connectionism during the "AI Winter," significant developments were occurring in hardware physics. In the late 1980s at Caltech, physicist Carver Mead—a pioneer of Very Large Scale Integration (VLSI) design—began to question the trajectory of digital computing.

Mead observed that while digital computers were becoming exponentially faster, they were also becoming less efficient in terms of energy per operation. He noted that using transistors as rigid, high-power switches to perform boolean logic was energetically wasteful compared to the biological systems they aimed to emulate.

In 1990, Mead published his seminal paper, *Neuromorphic Electronic Systems* [24], coining the term "neuromorphic" to describe hardware that mimics the biological structure of the nervous system. His thesis proposed that rather than simulating neural equations via software on digital computers, engineers should construct physical hardware that exploits the same physical laws as the biological nervous system.

The foundational insight of the field was the physical analogy between silicon physics and ion-channel physics. In standard digital electronics, transistors are operated in "strong inversion," driven by high voltages to act as binary switches. Mead realized that a single transistor, operating in its "subthreshold" region, follows the same exponential Boltzmann statistics that govern the flow of ions through biological channels.

This realization implied that a single transistor could physically compute the nonlinear functions used by biological neurons, but with significantly higher speed and lower power consumption. Consequently, synaptic functions could be implemented by single transistors rather than complex arrangements of logic gates.

To demonstrate this concept, Mead and his doctoral student Misha Mahowald developed the *Silicon Retina* in 1991 [25]. Unlike a standard camera, which captures full frames at fixed intervals (generating redundant data), the Silicon Retina operated asynchronously. It utilized analog circuits to compute spatial and temporal derivatives directly on-chip, outputting discrete "events" only when local light intensity changed.

This event-driven approach solved the redundancy problem inherent in frame-based sampling. If the scene remained static, the system transmitted no data and consumed negligible energy. This demonstrated that by aligning the hardware physics with the computational task, sensory information could be processed with a fraction of the power required by conventional digital systems.

Since the inception of neuromorphic computing, neuroscience has also advanced significantly. While Mead's early work was based on the physical intuition of the transistor, modern neuromorphic engineering now incorporates a richer understanding of neuronal dynamics, synaptic plasticity, and network architecture. To advance the field, we must combine these foundational hardware insights with the principles of modern mechanistic neuroscience.

3 • BIOLOGICAL PRINCIPLES

The biological brain remains the gold standard for energy-efficient, robust, and adaptive computation. Since the establishment of the Neuron Doctrine, modern neuroscience has uncovered the specific physical mechanisms that underpin this efficiency. To engineer systems that truly rival biological performance, we must transcend the highly simplified abstractions of early cybernetics. We cannot simply mimic the brain's output; we must emulate its internal dynamics. This requires viewing the neuron not as a static summing unit, but as it functions in reality: a complex, time-dependent, and event-driven processor.

This section provides a mechanistic overview of the nervous system, translating biological observations into the computational primitives required for neuromorphic engineering. It explores the structural hierarchy of the neuron, the physics of the action potential, and the mathematical models used to capture these dynamics in silicon.

3.1 • NEURON STRUCTURE & FUNCTION

In Section 2 we established the neuron as the fundamental computational unit of the brain. While it shares standard cellular machinery like mitochondria and a nucleus with other cells, it is morphologically specialized for information transmission. A neuron consists of three functional zones:

The Input (Dendrites): A branching tree structure that collects signals from thousands of upstream neurons. This is where inputs are integrated.

The Integration Zone (Soma): The cell body where electrical potentials from the dendrites summate.

The Output (Axon): A long, cable-like structure that transmits the neuron's own signal to downstream targets.

The neuron exhibits a distinct morphological polarization that dictates the direction of information flow. The process begins at the "dendritic arbor", a complex branching structure that maximizes the surface area for synaptic connectivity. These dendrites serve as the primary receptor sites, where neurotransmitters binding to post-synaptic terminals induce local conductance changes. These signals propagate passively toward the soma (cell body), the neuron's central processing unit. The soma acts as an integrator, spatially and temporally summing the incoming synaptic currents. Finally, the processed signal is transmitted via the axon, a singular, elongated projection. In many vertebrate neurons, the axon is insulated by a myelin sheath,

which facilitates saltatory conduction—a mechanism that allows high-speed signal propagation over long distances with minimal signal degradation.

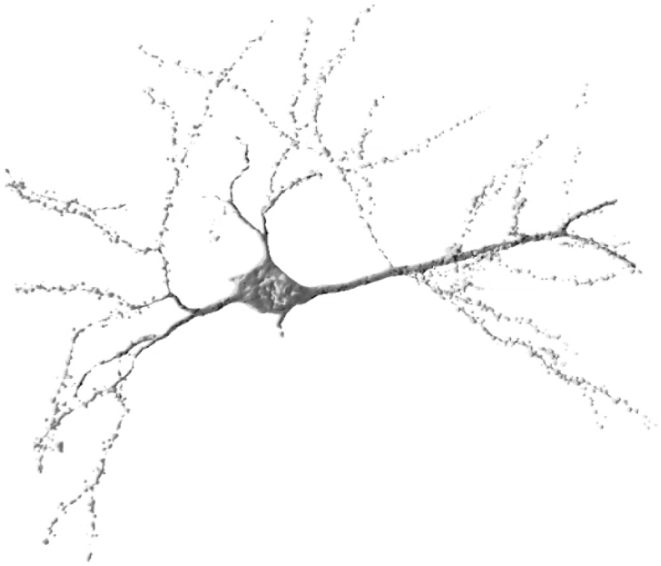

Figure 4: Image of a neuron, we the soma, axon and the densrites [26]

Functionally, the neuron operates as an electrochemical system enclosed by a cell membrane, known as the "lipid bilayer". This membrane is a thin, fatty structure that is impermeable to ions, acting as an electrical insulator. However, the fluids inside and outside the cell are conductive electrolytes. Consequently, the interaction between the insulating membrane and the conductive fluids creates a biological capacitor, capable of storing charge.

By actively pumping sodium (Na^+) out and potassium (K^+) in via the Na^+-K^+ ATPase pump, the cell maintains an electrochemical gradient across this capacitor, resulting in a stable "resting potential" of approximately -70 mV.

Computation occurs through the modulation of this voltage by competing synaptic inputs. Excitatory inputs cause ion channels to open, allowing positive ions to influx; this reduces the negative charge (depolarization) and pushes the potential toward the firing threshold. Conversely, inhibitory inputs activate channels for negative ions (like Chloride, Cl^-), driving the potential away from the threshold (hyperpolarization). The soma integrates these opposing push and pull signals. If the aggregate membrane potential surpasses a critical threshold (approximately -55 mV), the system undergoes a bifurcating phase transition. Voltage-gated sodium channels cascade open, triggering an action potential--a rapid, non-linear depolarization spike that propagates down the axon. This mechanism is governed by the "all-or-nothing" principle: the output is discrete and binary, effectively filtering out sub-threshold noise.

Immediately following a spike, the neuron enters a "refractory period" during which ion gradients are restored, imposing a hard limit on the maximum firing frequency and ensuring the temporal separation of events.

It is important to acknowledge that the biological brain exhibits significant cellular diversity beyond this idealized model. The nervous system contains non-neuronal cells known as “glia”, which provide structural support and manage energy delivery, though they are generally not considered direct participants in fast information transmission. Additionally, while the vast majority of cortical neurons communicate via uniform action potentials (spikes), certain sensory neurons utilize “graded potentials”, where the signal amplitude varies continuously. However, as spiking neurons represent the dominant computational paradigm for information processing in the cortex, this thesis focuses exclusively on the spiking model as the basis for neuromorphic emulation.

3.2 • ACTION POTENTIAL & SPIKE TRAINS

As established in the previous section, the action potential is an “all-or-nothing” event. It serves as the fundamental mechanism by which neurons transmit information. Crucially, the waveform of this event is stereotypical: for a given neuron, every spike exhibits a nearly identical amplitude and duration (typically 1–2 ms), independent of the input intensity that triggered it.

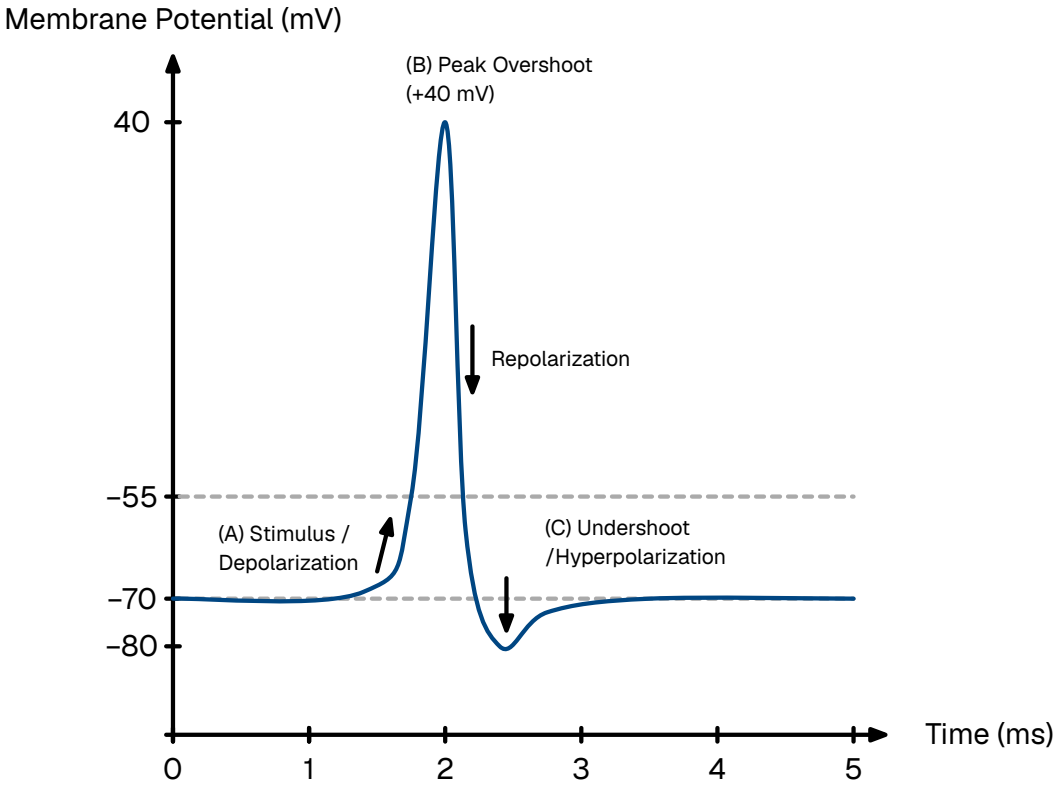

Figure 5: The phases of a typical neuronal action potential. (A) An incoming stimulus depolarizes the membrane past the threshold (−55 mV), triggering a rapid spike. (B) The membrane potential reaches a peak overshoot (+30 mV) before repolarizing. (C) A temporary undershoot (hyperpolarization) occurs before returning to the resting state (−70 mV). The neuron cannot fire during the absolute refractory period (D) and requires a stronger stimulus to fire during the relative refractory period (E).

This biological invariance permits a fundamental simplification in neuromorphic modeling:

> If the spike waveform is invariant across neurons and time, the waveform itself carries no information. Consequently, the information content of the signal is encoded entirely in the precise timing of the spike.

To model this mathematically, we abstract the continuous biophysical voltage trace into a dimensionless point process. We treat the action potential not as a function of voltage over time, but as a singular event occurring at a precise instant, t_f, with negligible duration. The standard tool for this abstraction is the Dirac delta function denoted as, $\delta(t)$.

The Dirac delta is a generalized distribution defined by the property that it is zero everywhere except at the origin, yet integrates to unity. This represents an idealized pulse of infinite height and zero width, containing a finite unit of effect.

$$\delta(t) = \begin{cases} \infty \text{ if } t = 0 \\ 0 \text{ if } t \neq 0 \end{cases}, \quad \int_{-\infty}^{+\infty} \delta(t)\,\mathrm{d}t = 1$$

Equation 1: The defining properties of the Dirac delta function.

Under this formalism, the output of a neuron is modeled not as a continuous signal, but as a "spike train"—a temporal sequence of these Dirac impulses [27]. For a neuron emitting N spikes at times $\left\{t^{(1)}, t^{(2)}, ..., t^{(N)}\right\}$, the output signal $S(t)$ is defined as:

$$S(t) = \sum_{f=1}^{N} \delta\left(t - t^{(f)}\right)$$

Equation 2: A spike train represented as a sum of Dirac delta functions.

This abstraction allows the post-synaptic effect to be modeled using linear systems theory. In neuron models that use this framework, the interaction is treated as instantaneous charge deposition: the arrival of a delta function $\delta\left(t - t_f\right)$ imparts a discrete step-change to the post-synaptic current. This mimics the rapid opening of ion channels without requiring the computational overhead of simulating the complex voltage trajectory. The shift from continuous values to discrete spike trains fundamentally alters the computational paradigm, moving from spatial representations (magnitude-based) to spatio-temporal representations (time-based).

3.3 • NEURON MODELS

In the quest to simulate the brain, there exists a fundamental trade-off between biological realism and computational efficiency. At the high end of the spectrum lie conductance-based models, most notably the Hodgkin-Huxley model. This formalism describes the neuron not as a simple computational unit, but as an electrical

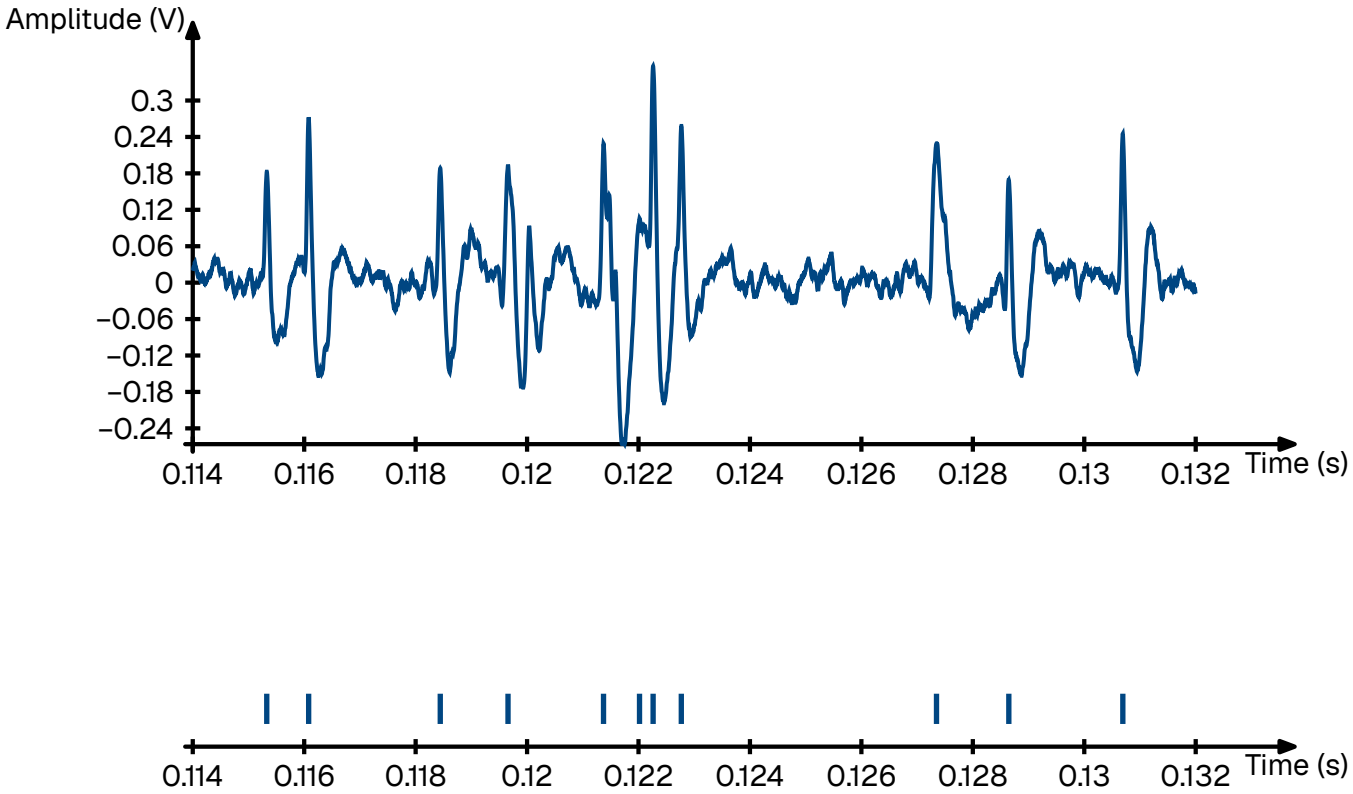

Figure 6: Transformation of continuous membrane voltage (top) into a discrete spike train (bottom). The membrane voltage trace is a recording from the L5 dorsal rootlet of a rat using a multiple electrode array [28]

circuit with variable resistors representing the precise, non-linear opening and closing dynamics of specific ion channels (sodium, potassium, leak) [29].

Large-scale initiatives, such as the Blue Brain Project, utilize even more granular "multi-compartment" models. These simulations treat the neuron as a complex 3D structure, discretizing the dendritic arbor and axon into hundreds of segments to model how current flows through the specific morphology of the cell [30]. While invaluable for pharmacological research, these models are computationally prohibitive for large-scale neuromorphic engineering. Simulating a mere second of biological time for a small network using these equations requires supercomputing resources.

To build practical, scalable neuromorphic hardware, we must abstract these biophysical details into a phenomenological model. We seek a mathematical framework that captures the essential computational properties—integration, leakage, and thresholding—without simulating the underlying molecular physics.

3.3.1 • THE LEAKY INTEGRATE-AND-FIRE (LIF) MODEL

The standard approximation used in neuromorphic engineering is the Leaky Integrate and Fire (LIF) model [27]. This framework aligns perfectly with the "point process" abstraction established in the previous section, as it treats action potentials as instantaneous, discrete events. Its state is defined by a single scalar variable,

the membrane potential $u(t)$. The sub-threshold dynamics are governed by a linear differential equation analogous to a simple RC (Resistor-Capacitor) circuit. We implement this model as the computational baseline for our visual classification tasks in Section 6.3.2 (Model B).

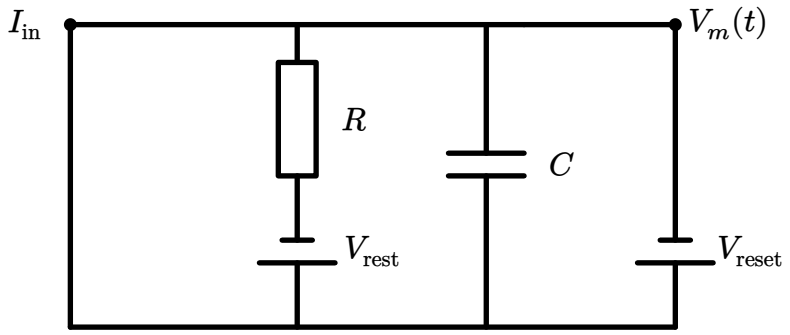

Figure 7:

$$\tau_m \frac{\mathrm{d}u}{\mathrm{d}t} = -(u - u_{\text{rest}}) + RI(t)$$

Equation 3: The Leaky Integrate-and-Fire (LIF) differential equation. The change in voltage is driven by the leak (decay to rest) and the input current.

Where τ_m is the membrane time constant (determining how fast the neuron "forgets"), u_{rest} is the resting potential, R is the membrane resistance, and $I(t)$ is the input current.

Connecting this to the spike train abstraction derived in the previous section, the input current I(t) is not continuous. It is a sequence of discrete events arriving from pre-synaptic neurons j with weight w_j. Mathematically, this is modeled as a sum of Dirac delta functions:

$$I(t) = \sum j w_j \sum f \delta\left(t - t_{j(f)}\right)$$

Equation 4: Synaptic input modeled as a weighted sum of Dirac delta functions.

Because the differential equation is linear below the threshold, we can solve it analytically. The membrane potential $u(t)$ becomes a convolution of the input spike train with the system's impulse response (a decaying exponential kernel). This means the potential at any moment is simply the sum of the decaying traces of all past spikes:

$$u(t) = u_{\text{rest}} + \sum j w_j \sum f \exp\left(-\frac{t - t_{j(f)}}{\tau_m}\right)$$

Equation 5: The analytical solution for the membrane potential. The current voltage is the superposition of all past inputs, decayed by time constant τ_m.

The differential equation above describes the continuous sub-threshold dynamics. To complete the model, we must define the discrete "Fire" mechanism. The LIF neuron operates as a hybrid dynamical system: it integrates continuously until a discontinuity is triggered.

When the membrane potential $u(t)$ exceeds a specific threshold voltage ϑ, the neuron emits a spike (a Dirac delta event). Immediately following this event, the membrane potential is not governed by the differential equation but is forced to a reset value, u_{reset}.

To mimic the biological limit on firing frequency, the model enforces an absolute refractory period, Δ_{ref}. After a spike occurs at time t_f, the neuron is clamped to the resting potential for a duration of Δ_{ref}. During this interval, the differential equation is suspended; the neuron ignores all inputs and cannot fire, regardless of the stimulus intensity.

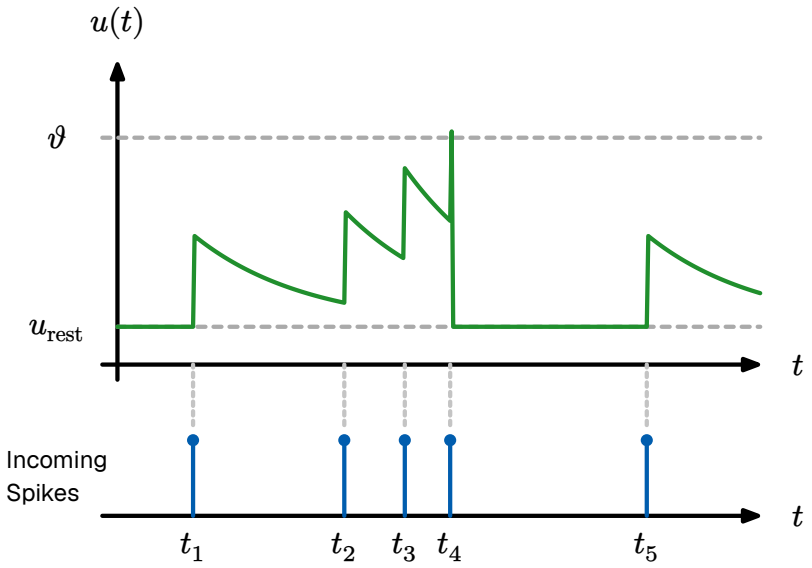

Figure 8: The dynamics of an Leaky Integrate and Fire (LIF) neuron. (A) The membrane potential integrates inputs. (B) Upon crossing the threshold ϑ, a spike is emitted and the voltage is reset. (C) The voltage is clamped during the refractory period.

Mathematically, this firing condition is expressed as:

$$\text{If } \; u(t) > \vartheta \rightarrow \begin{cases} \text{Emit spike:} \;\; S(t) \rightarrow S(t) + \delta(t) \\ \text{Reset voltage:} \;\; u(t) \rightarrow u_{\text{reset}} \\ \text{Pause integration for} \;\; t \in \left(t_f, t_f + \Delta_{\text{ref}}\right] \end{cases}$$

Equation 6: The discrete firing and reset condition.

This equation represents the engine of most neuromorphic algorithms. It defines a system that integrates information over time and leaks it to ensure temporal relevance. However, once $u(t)$ crosses a threshold ϑ, the linearity breaks and the neuron emits a spike and $u(t)$ is manually reset to u_{reset}.

3.3.2 • THE GENERALIZED (ADAPTIVE) LIF MODEL

While the standard LIF model is computationally efficient, its one-dimensional nature limits it primarily to tonic spiking (regular firing under constant input). It struggles to replicate the complex, non-linear behaviors observed in the cortex, such

as bursting (clusters of rapid spikes) or spike-frequency adaptation (slowing down after sustained activity).

To capture these dynamics without reverting to the computationally heavy Hodgkin-Huxley equations, we employ the Generalized Leaky Integrate and Fire (GLIF) model [27]. This extends the system by introducing a second state variable, $w(t)$, representing cellular adaptation.

$$\tau_m \frac{\mathrm{d}u}{\mathrm{d}t} = -(u - u_{\text{rest}}) + RI(t) - w$$
$$\tau_w \frac{\mathrm{d}w}{\mathrm{d}t} = a(u - u_{\text{rest}}) - w$$

Equation 7: The Adaptive GLIF system. The adaptation variable w provides negative feedback, enabling complex dynamics like bursting and adaptation.

In this coupled system, w provides a negative feedback loop. Every time the neuron spikes, w increments by a constant b, acting as a physiological drag on the membrane potential. By adjusting the coupling parameters between u and w, this two-dimensional system can be tuned to emulate the full spectrum of biological firing patterns.

It is natural to question whether such a mathematically reduced model can genuinely capture the behavior of biological neurons. While the GLIF model discards the specific ionic mechanisms of the Hodgkin-Huxley equations, empirical validation demonstrates that it retains superior computational dynamics for large-scale modeling.

In the 2008 *Quantitative Single-Neuron Modeling Competition* organized by the INCF, phenomenological models like the GLIF (specifically the Adaptive Exponential Integrate-and-Fire) unexpectedly outperformed highly detailed biophysical models in predicting the precise spike times of real cortical neurons.

This counter-intuitive success is due to parameter sensitivity. Complex conductance-based models have dozens of unobservable parameters that are difficult to tune. In contrast, the GLIF model captures the "net effect" of these mechanisms using macroscopic parameters that can be robustly fitted to data. As demonstrated by Izhikevich (2003) [31], this simple system of two differential equations is capable of reproducing all known firing patterns observed in the mammalian cortex [32]. We leverage this state-dependent adaptation to implement Model D (Section 6.3.2), evaluating its effectiveness in temporal sequence decoding.

Consequently, for the purpose of neuromorphic engineering, the GLIF model represents the optimal trade-off between biological fidelity and computational efficiency.

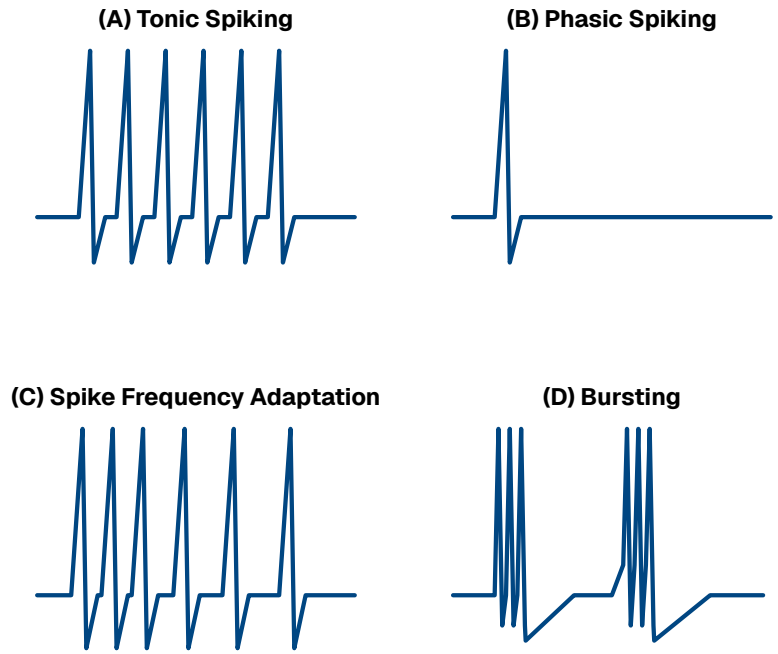

Figure 9: The Generalized Leaky Integrate and Fire (GLIF) model is capable of reproducing the diverse firing patterns of biological cortical neurons, as categorized by Izhikevich (2003) [31].

3.4 • NEURAL CODING

In classical digital computing, information is represented by combining bits into richer structures, such as floating-point or integer numbers. For instance, the luminance of a pixel is typically stored as a discrete 8-bit or 32-bit integer. Conversely, analog electronics represent values as continuous currents or voltages, offering infinite resolution within the dynamic range of the hardware.

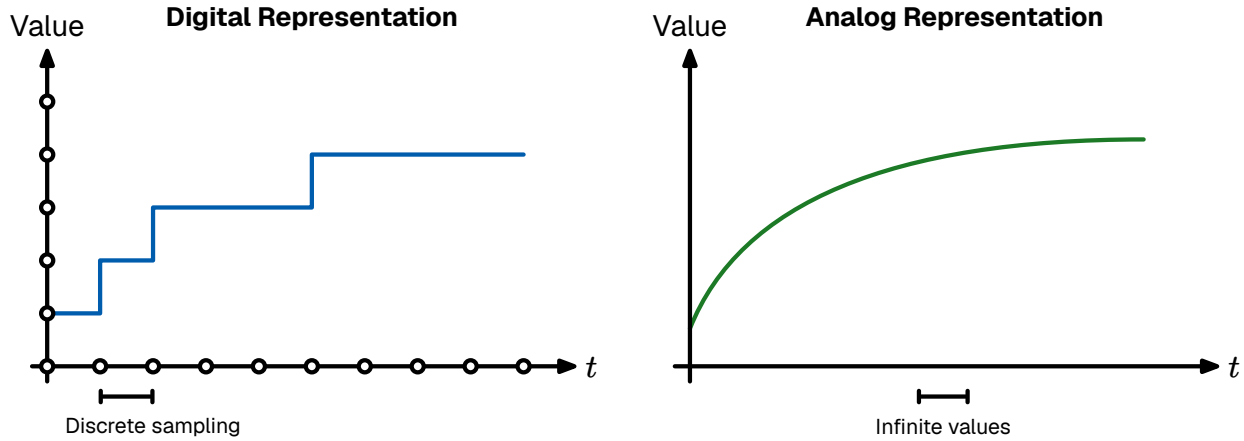

Figure 10: Digital left analog right representation

The biological brain occupies a unique middle ground. While neurons operate using analog membrane potentials, their communication output—the action potential—is discrete and binary. As established in Section 3.2, the waveform of a spike is stereotypical; it looks like a "digital bit" in amplitude. However, unlike a digital computer which is synchronized to a rigid clock, these spikes occur in continuous time. There-

fore, the information in the nervous system is not stored in the shape of the signal, but in the structure of the spike train itself.

Deciphering the "Neural Code"—the set of rules by which sensory stimuli are translated into these spike sequences—remains one of the central challenges in neuroscience. Currently, several coding schemes are hypothesized to coexist, each offering different trade-offs between latency, information density, and robustness.

3.4.1 • RATE CODING

The most traditional interpretation of neural activity is rate coding. In this paradigm, information is conveyed by the mean firing frequency of a neuron over a specific temporal window. A strong stimulus (e.g., high pressure on skin) elicits a high firing rate, while a weak stimulus results in sparse activity.

This model effectively treats the neuron as an Analog-to-Digital converter where the precise timing of individual spikes is treated as noise; only the average count carries the signal. While rate coding is robust and easily observed in motor neurons, it suffers from a fundamental latency barrier. To estimate a rate with reasonable precision, the post-synaptic neuron must integrate spikes over a significant duration (tens or hundreds of milliseconds). This contradicts the rapid reaction times (often < 100 ms) observed in biological agents, suggesting that rate coding alone cannot account for time-critical processing @ [33].

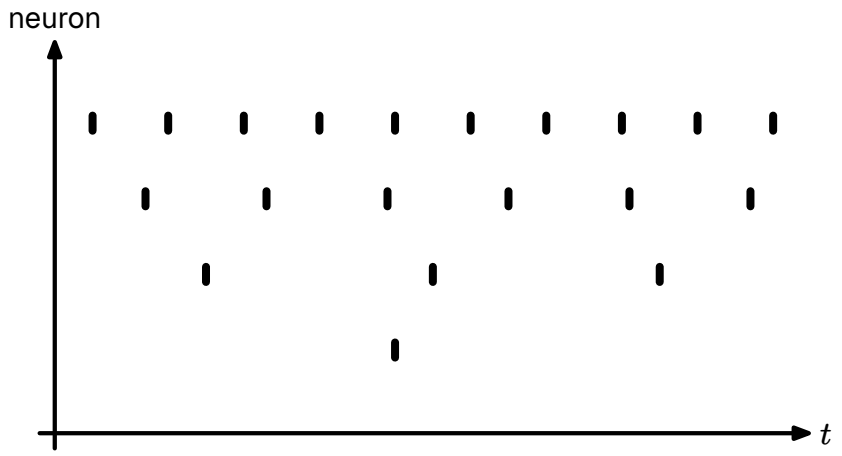

Figure 11: Rate Coding: The stimulus intensity is encoded in the frequency of the spike train. Stronger stimuli elicit more spikes per second.

3.4.2 • TEMPORAL CODING

To explain the speed of biological processing, neuromorphic engineering emphasizes temporal coding. In this regime, the precise timing of a spike carries significant information. A primary example is TTFS coding.

In a TTFS scheme, the intensity of a stimulus is inversely mapped to the latency of the response relative to a stimulus onset [34]. A stronger input causes the neuron to integrate and cross the threshold faster, firing earlier than neurons receiving weak

inputs. This shifts the computational model from counting spikes to a "race" between spikes.

In a network utilizing lateral inhibition (Winner-Take-All (WTA)), the first neuron to fire inhibits its neighbors, allowing a decision to be made as soon as the first meaningful bit of data arrives. This eliminates the need to wait for a time window to close, drastically reducing latency. Furthermore, since TTFS coding prioritizes the strongest signals, it acts as a natural filter: the most prominent features arrive first, allowing the system to process signal over noise. This TTFS scheme forms the primary encoding modality for our MNIST experiments, as detailed in Section 6.3.1.

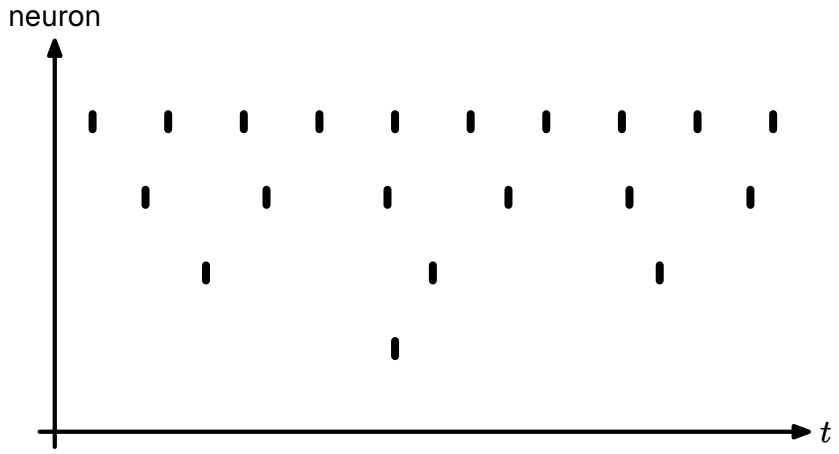

Figure 12: Temporal Codiing (Time-To-First-Spike (TTFS)): Stimulus intensity is encoded in the latency of the response. Stronger inputs (I_1) trigger an earlier spike (t_1) compared to weaker inputs (I_2).

3.4.3 • THE PHASE AMBIGUITY PROBLEM

A critical challenge in temporal coding is the need for a temporal reference frame. In Rate Coding, the "phase" (absolute timing) is irrelevant. However, in Temporal Coding, a spike at time t only has meaning relative to a reference t_0. If the receiver does not know when the stimulus started, it cannot decode the latency.

In engineering, this is solved by a global clock or a "frame start" signal. In the brain, evidence suggests that background oscillatory rhythms (brain waves, such as theta or gamma cycles) may serve as this global reference, allowing populations of neurons to synchronize their "clocks" and decode relative timings accurately.

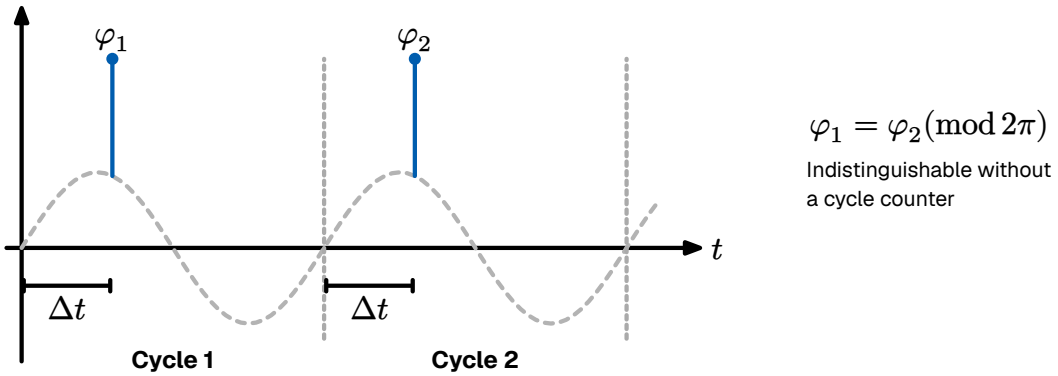

Figure 13: The phase ambiguity problem in temporal encoding. Spikes occurring at the same relative phase (φ_1 and φ_2) across different oscillation cycles are mathematically indistinguishable ($\varphi_1 = \varphi_2 (\mathrm{mod}\, 2\pi)$). Without a mechanism to track the global cycle count, downstream neurons cannot determine whether a spike represents a delayed response to a previous stimulus or an early response to a new one.

3.4.4 • POPULATION & SPARSE CODING

While single-neuron codes provide the basic signaling mechanism, the brain employs ensemble strategies to ensure robustness and precision. In population coding, variables are represented by the joint activity of a large group of neurons, each with broad, overlapping tuning curves. A classic example is found in the Primary Visual Cortex (V1), where orientation-selective neurons each respond maximally to a preferred angle but also fire weakly for nearby orientations. By reading the weighted population vector across the group, the network reconstructs the stimulus with far greater precision than any individual cell could provide alone. The brain further optimizes for metabolic efficiency through sparse coding, where only a small fraction of neurons are active at any moment. This strikes a mathematical balance between representational capacity and energy cost, and is naturally enforced by lateral inhibition circuits that suppress weaker, competing responses.

3.4.5 • COEXISTENCE OF CODES

These schemes are not mutually exclusive but complementary. A circuit may use TTFS for a rapid initial response—alerting the system to a salient change—before transitioning to rate-based activity for sustained processing. Neuromorphic systems often adopt this hybrid approach, using temporal codes for energy-efficient sparse event transmission and rate-based readouts for interfacing with downstream control systems. This thesis follows the same principle, using TTFS encoding for the transmission of visual features combined with a population-level representation at the hidden layer.

3.5 • NEURAL NETWORKS

Having established the mathematical description of the individual neuron, we now turn to the collective behavior of these units. A single neuron, regardless of its dynamical complexity, is of limited computational utility in isolation. Functional intelligence emerges only when these units are organized into specific structural topologies. We implement these topologies in our experimental framework, as detailed in Section 6.2.

The brain is not a random mesh of connections; it is constructed from recurring architectural "motifs" that appear across various cortical areas. Understanding these motifs is essential for designing neuromorphic systems that transcend simple feed-forward processing.

3.5.1 • SYNAPTIC EFFICACY & WEIGHTS

Before analyzing the structural topology of networks, we must define the fundamental unit of connectivity: the synapse. In the biological brain, neurons do not touch; they are separated by a microscopic gap known as the synaptic cleft. Communication across this gap is chemical, mediated by the release of neurotransmitters.

The efficiency of this transmission—determined by factors such as the amount of neurotransmitter released and the number of post-synaptic receptors—is abstracted in mathematical models as the synaptic weight (w).

In the SNN formalism, the weight represents a scaling factor for the incoming spike. When a pre-synaptic neuron j fires a spike at time t_j, it induces a Post Synaptic Current (PSC) in neuron i scaled by the weight w_{ij}. Mathematically, the total synaptic input $I(t)$ is the weighted sum of all incoming spike trains:

$$I_{i(t)} = \sum_j w_{ij} \cdot S_{j(t)}$$

Equation 8: The synaptic input current as a weighted sum of incoming impulses.

Synaptic weights determine not just the magnitude but also if the synapse is excitatory or inhibitory.

Excitatory Synapses ($w > 0$): These depolarize the target neuron, pushing its membrane potential closer to the firing threshold (e.g., Glutamate synapses).

Inhibitory Synapses ($w < 0$): These hyperpolarize the target neuron, pushing the potential away from the threshold (e.g., GABA synapses).

A fundamental constraint in biological networks, known as Dale's Principle, states that a neuron performs the same chemical action at all of its synaptic outputs [35]. This means a neuron is strictly excitatory or strictly inhibitory; it cannot send positive signals to one neighbor and negative signals to another. While standard ANNs

often violate this rule for mathematical convenience (allowing weights to flip signs during training), bio-plausible neuromorphic architectures often enforce this constraint to mimic the distinct populations of Pyramidal (excitatory) and Interneuron (inhibitory) cells found in the cortex.

The network must maintain a precise Excitation-Inhibition (E/I) Balance. The brain operates at a critical point of instability:

Excess Excitation leads to runaway feedback loops (analogous to epileptic seizures).

Excess Inhibition leads to signal extinction (quiescence).

3.5.2 • DIRECTIONALITY

Structurally, neural topologies can be categorized by the flow of information.

In sensory peripheries (such as the retina) and early processing stages, information flows unidirectionally from input to output. This topology supports rapid, reflex-like feature extraction. This configuration is known as a feed-forward network, which is mathematically equivalent to a Directed Acyclic Graph (Directional Acyclic Graph (DAG)) and serves as the standard architecture for most Deep Learning CNNs.

In higher cognitive areas, the dominant topology is recurrence. Neurons form feedback loops, connecting back to themselves or to distinct layers. This recurrence introduces a time component to the computation, transforming the network into a dynamical system where the current output depends not only on the input but on the network's previous state (history).

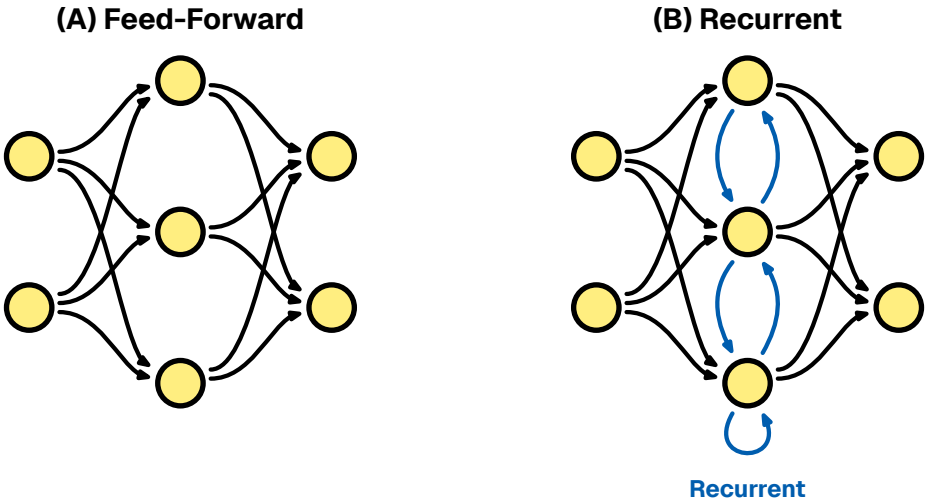

Figure 14: Network topologies. (A) Feed-Forward. (B) Recurrent.

3.5.3 • SYNAPTIC HYPOTHESIS: STRUCTURE AS FUNCTION

A foundational premise in neuromorphic engineering, derived from biological observation, is that the neuron operates largely as a generic processing unit. The functional identity of a neural circuit—whether it processes visual stimuli or governs

motor control—is determined principally by the topology and efficacy of its synaptic interconnections.

This paradigm, known as the Synaptic Hypothesis, posits that the physical configuration of synaptic weights constitutes the substrate for all computation and memory. Unlike traditional Von Neumann architectures, where data is retrieved from a distinct memory module and processed in a central CPU, biological systems eliminate the distinction between "data" and "program." Memory is not a static artifact, but a latent computational potential distributed across the network's structural graph. Consequently, learning in a neuromorphic system is realized through the physical alteration of these synaptic weights, ensuring robust, decentralized processing that is inherently resistant to localized hardware failure (graceful degradation).

3.5.4 • INHIBITION PATTERNS

A ubiquitous micro-circuit motif in the cortex is lateral inhibition. In this configuration, an active excitatory neuron stimulates distinct inhibitory interneurons, which in turn suppress the activity of neighboring excitatory neurons. This competition engenders a WTA dynamic: as one neuron—representing a specific feature or decision—becomes active, it effectively silences its competitors. In the context of neuromorphic engineering, WTA circuits are indispensable; they provide a physical mechanism for both noise reduction, by actively suppressing weak, sub-threshold signals, and categorical decision making, enabling the circuit to autonomously select the most salient option without the need for a central processor to sort or compare values. We utilize this WTA motif in our output layer to enforce categorical decision-making.

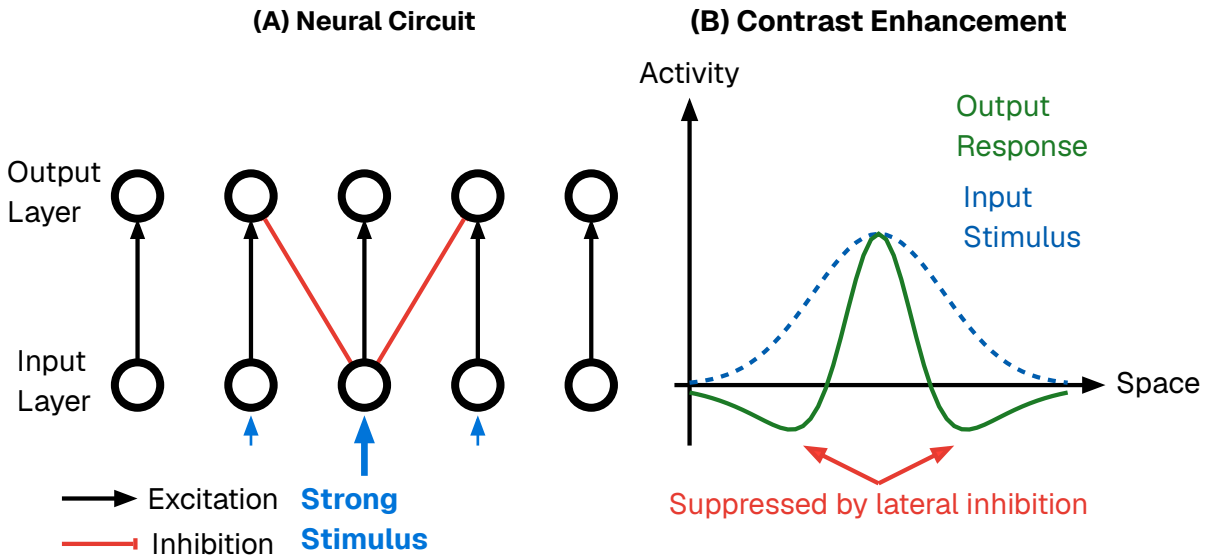

Figure 15: The mechanism of lateral inhibition. (A) A highly stimulated neuron in the input layer strongly excites its corresponding output neuron while simultaneously sending lateral inhibitory signals to its immediate neighbors. (B) This architectural motif acts as a spatial filter, producing a contrast enhancement effect. A broad input stimulus (dashed blue line) is transformed into a sharper output response (solid purple line) characterized by an amplified center and suppressed surroundings (a "Mexican hat" profile), thereby sharpening signal boundaries.

While lateral inhibition processes information in the spatial domain, Feed-Forward Inhibition (FFI) operates in the temporal domain. Structurally, this motif bifurcates an input signal into two parallel pathways: a direct excitatory route to the target neuron, and a disynaptic inhibitory route that reaches the same target with a slight synaptic delay. This architecture creates a narrow "temporal window of opportunity." Because the excitation triggers the neuron immediately before the delayed inhibition abruptly truncates the response, the neuron is prevented from integrating noise over extended durations. Consequently, FFI forces the neuron to function as a precise Coincidence Detector rather than a sluggish integrator, a dynamic that is fundamental to sound localization in the auditory cortex and fine-grain timing in the somatosensory system.

3.6 • BIOLOGICAL LEARNING

As previously established, the functional identity of a neural circuit is not defined by a transient software state, but by its physical hardware configuration. Consequently, "learning" in a biological substrate cannot be viewed as a simple parameter optimization; it is a fundamental morphological process. If structure dictates function, then the acquisition of new skills or memories necessitates the physical restructuring of the connectome itself.

Because the brain lacks a central supervisor or global communication bus, this restructuring must be driven by Locality. A synapse can only change based on

information physically available at the cleft: the activity of the pre-synaptic axon, the voltage of the post-synaptic dendrite, and the immediate neurochemical environment. Despite this constraint, the brain successfully credits specific synaptic events with outcomes that occur seconds or minutes later.

This adaptation occurs across multiple timescales and spatial resolutions via two distinct mechanisms: Structural Plasticity (the rewiring of the network topology) and Synaptic Plasticity (the modulation of connection strength).

3.6.1 • STRUCTURAL PLASTICITY

While synaptic weight adjustment accounts for rapid learning and pattern recognition, the long-term storage of information and the optimization of energy efficiency are governed by structural plasticity. This mechanism involves the physical genesis (synaptogenesis) and removal (pruning) of synapses and even entire axonal branches.

Synaptogenesis: When neurons are repeatedly co-active but lack a direct connection, the brain can physically grow new dendritic spines and axonal boutons to bridge the gap. This effectively alters the network's topology, creating new pathways for information flow where none existed before.

Pruning: Equally critical is the removal of redundant or noisy connections. During sleep and developmental critical periods, the brain aggressively prunes weak synapses. This "sparsification" reduces metabolic cost and increases the signal-to-noise ratio of the circuit by removing irrelevant pathways.

In the context of the Synaptic Hypothesis, structural plasticity represents the "compiling" of temporary associations into permanent hardware architecture.

3.6.2 • SYNAPTIC PLASTICITY

Once a structural connection exists, its efficacy—the magnitude of the post-synaptic response to a pre-synaptic spike—must be tuned. In biological terms, this "weight" corresponds to the amount of neurotransmitter released and the density of receptors on the receiving side. This fine-grained adjustment is governed by local learning rules.

3.6.3 • HEBBIAN LEARNING: RATE-BASED CORRELATION

The foundational axiom of biological learning was postulated by Donald Hebb in 1949. Hebb proposed that synaptic efficiency is a function of the correlated activity between two neurons. Colloquially summarized as "Neurons that fire together, wire together," this rule implies that the brain learns by detecting statistical regularities in sensory input.

Mathematically, if neuron A consistently takes part in firing neuron B, the connection from A to B is strengthened. This mechanism allows the brain to perform unsupervised clustering, physically encoding associations between features that occur simultaneously in the environment (e.g., the smell of smoke and the sight of fire).

3.6.4 • SPIKE-TIMING-DEPENDENT PLASTICITY (STDP)

Modern neuroscience has refined Hebb's macroscopic theory into a precise, millisecond-scale mechanism known as Spike-Timing-Dependent Plasticity (STDP). Unlike rate-based models, STDP operates on the precise timing of individual action potentials, introducing the critical element of causality.

The STDP rule adjusts the synaptic weight based on the relative timing difference (Δt) between the pre-synaptic input and the post-synaptic output:

Long-Term Potentiation (LTP): If the input spike arrives **before** the output spike ($\Delta t > 0$), it implies the input contributed to the firing. The synapse is strengthened to reinforce this causal link.

Long-Term Depression (LTD): If the input spike arrives **after** the output spike ($\Delta t < 0$), the input was irrelevant to the decision. The synapse is weakened.

This asymmetry allows the network to self-organize, naturally filtering out random noise while reinforcing specific spatiotemporal patterns. We adapt this causal rule to a discrete temporal window for our unsupervised learning experiments in Section 6.4.2.

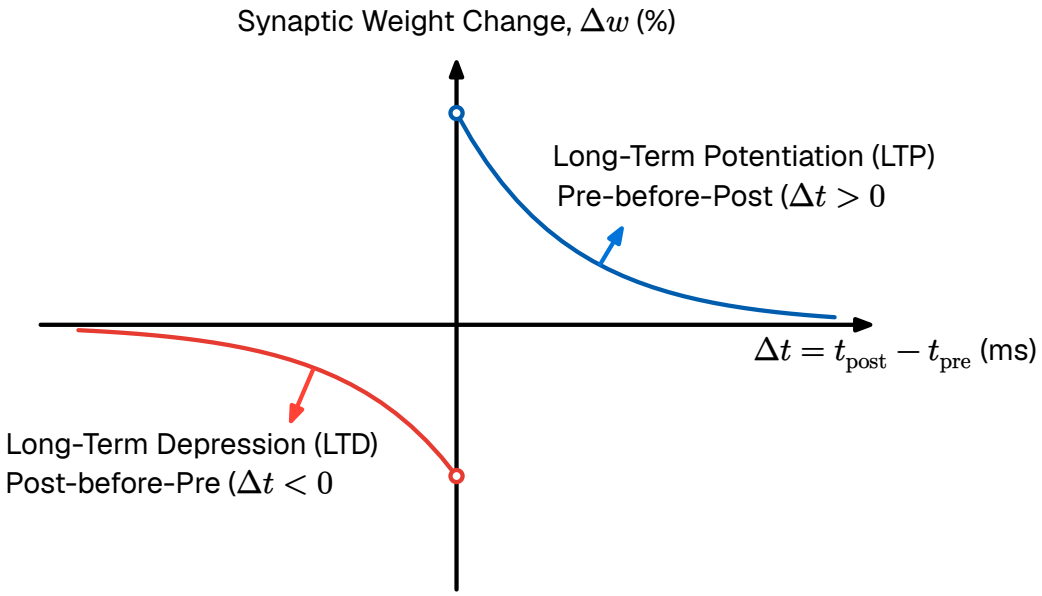

Figure 16: The Spike-Timing-Dependent Plasticity (STDP) Learning Curve. Synaptic weight change is plotted against spike timing difference. Pre-before-post timing triggers strengthening Long-Term Potentiation (LTP), while post-before-pre triggers weakening Long-Term Depression (LTD).

3.6.5 • HOMEOSTATIC PLASTICITY

If Hebbian mechanisms (LTP) were the sole drivers of plasticity, neural networks would be inherently unstable. A positive feedback loop would emerge where strengthened synapses drive higher firing rates, which in turn induce further strengthening, leading to runaway excitation (seizures). Conversely, unchecked LTD could silence a network entirely.

To maintain stability, the brain employs Homeostatic Plasticity (or Synaptic Scaling). This is a global regulatory mechanism that operates on a slower timescale (minutes to hours). It functions as a negative feedback loop: if a neuron's average firing rate exceeds a target set-point, the cell chemically downscales the strength of all its incoming synapses. This ensures that neurons remain within a sensitive dynamic range, preventing saturation regardless of how strong the inputs become.

The following chapter shifts perspective—from biology to engineering. We examine the mathematical framework of modern Deep Learning, to identify where its abstractions diverge from biological reality and what computational cost those divergences impose. This analysis will make explicit the bottlenecks that neuromorphic architectures are designed to resolve, grounding the methodological choices of this thesis in a concrete technical rationale.

4 • TECHNICAL DETAILS OF MACHINE LEARNING

This chapter delineates the technical foundations of modern artificial intelligence, contrasting the established paradigms of Deep Learning (DL) with the emerging principles of Neuromorphic Engineering. We begin by analyzing the algorithmic architecture of standard Deep Learning, identifying the computational bottlenecks inherent in its reliance on dense matrix multiplication and backpropagation.

A critical distinction must be drawn between biological plausibility and bio-inspired engineering. From an engineering perspective, the primary objective is functional utility. An engineer may treat the brain merely as a source of heuristic inspiration rather than a blueprint to be copied dogmatically. However, the pursuit of biologically plausible systems remains vital; it offers potential advantages in robustness and energy efficiency while serving as a verification tool for neuroscience.

4.1 • OPTIMIZATION

Optimization is the selection of a "best candidate" with regard to defined criteria. Biological learning fits this description, where the optimal candidate is the configuration of synaptic weights that performs well for a specific task. Therefore, it is useful to establish a mathematical framework for this process.

Fundamentally, a deep learning model operates as a function approximator. We assume the existence of an unknown underlying function $f : X \to Y$ that perfectly maps inputs to their target outputs. Since this true function is unknown, we construct a family of hypothesis functions $f_{\boldsymbol{\theta}}(\boldsymbol{x})$ to approximate it. Here, $\boldsymbol{\theta} \in \mathbb{R}^d$ represents the state of the system—a vector containing all tunable parameters, such as synaptic weights or biases. The dimensionality d corresponds to the degrees of freedom of the model.

The key problems in optimization are defining the objective goal (the loss function) and finding the parameter configuration $\boldsymbol{\theta}$ that achieves that goal.

4.1.1 • SUPERVISED LEARNING

To guide the search for optimal parameters $\hat{\boldsymbol{\theta}}$, we must quantify the divergence between the model's predictions and the ground truth. We define a scalar Loss Function $\mathcal{L}(\hat{\boldsymbol{y}}, \boldsymbol{y})$ that evaluates the error on a single data point. To ensure generalization, we seek to minimize the Empirical Cost Function $J(\boldsymbol{\theta})$, defined as the average loss over a dataset of size N:

$$J(\boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}(f_{\boldsymbol{\theta}}(\boldsymbol{x}_i), \boldsymbol{y}_i)$$

Geometrically, the cost function $J(\boldsymbol{\theta})$ induces an Optimization Landscape. Finding a low-energy state in this non-convex topology is the central challenge of AI training. We rely on iterative optimization algorithms, principally Gradient Descent. This method updates the system state in the direction opposite to the gradient vector $\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$ (the steepest ascent). The update rule for iteration t is:

$$\boldsymbol{\theta}_{t+1} \leftarrow \boldsymbol{\theta}_t - \eta \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}_t)$$

Here, η represents the Learning Rate. Because computing the gradient over the entire dataset N is computationally prohibitive, modern AI employs Stochastic Gradient Descent (SGD), approximating the gradient using small random subsets (mini-batches). This introduces beneficial noise, preventing the system from getting trapped in shallow local minima.

Crucially, gradient descent requires the loss function to be differentiable. As will be discussed later, this presents a significant challenge for optimizing neuromorphic systems utilizing discrete, non-differentiable spike trains.

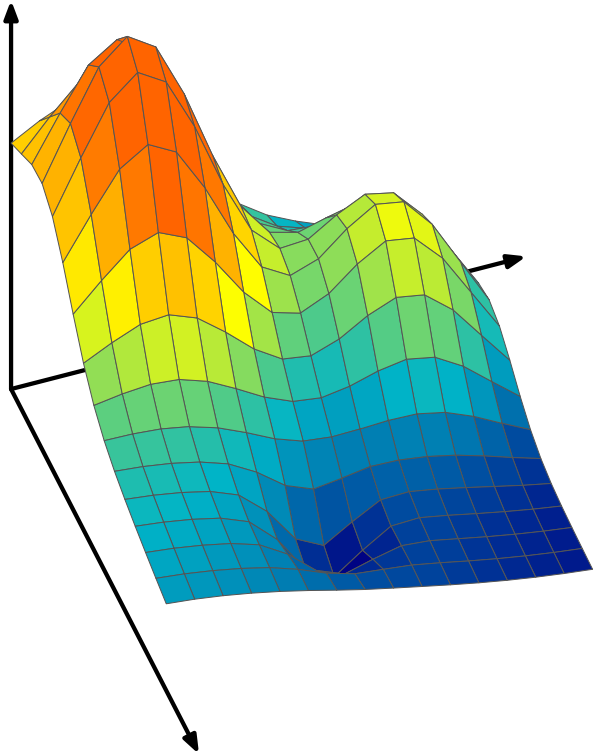

Figure 17: The Optimization Landscape. The system seeks to traverse the high-dimensional surface defined by $J(\boldsymbol{\theta})$ to find the global minimum $\boldsymbol{\theta}^*$, using the gradient ∇J as a navigational compass.

Strictly minimizing the empirical cost carries the risk of overfitting—the model memorizes training data including noise rather than learning the underlying function. In biological systems this is naturally regulated by metabolic constraints; the

brain prunes weak connections to maintain a sparse topology, effectively trading model complexity for generalization. In artificial systems this is managed via explicit regularization penalties added to the cost function.

4.1.2 • UNSUPERVISED LEARNING

While supervised learning relies on labeled targets, biological systems predominantly learn via Unsupervised Learning. In this regime, the dataset consists only of input vectors $X = \{\boldsymbol{x}_1, ..., \boldsymbol{x}_N\}$. The optimization objective shifts from minimizing prediction error to minimizing representation error.

Mathematically, the goal is often to discover a lower-dimensional manifold that efficiently captures the structure of the data. A common formulation is the minimization of Reconstruction Loss, where the system attempts to compress the input into a latent code and reconstruct it:

$$J(\boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^{N} \| \boldsymbol{x}_i - f_{\text{decode}\left(f_{\text{encode}(\boldsymbol{x}_i;\boldsymbol{\theta})}\right)} \|^2$$

Alternatively, the system may optimize for clustering density or distances between feature centroids. The distinction between supervised and unsupervised learing is critical for Neuromorphic Engineering, as biological plasticity rules (like STDP) are unsupervised, functioning by detecting statistical correlations in the input stream to build internal representations without external labels.

4.2 • DEEP LEARNING FRAMEWORK

Modern Deep Learning aggregates simple units into high-dimensional layers. A deep network with L layers is expressed as a composite function mapping input $\boldsymbol{x}$ to output $\boldsymbol{y}$ through nested operations:

$$\boldsymbol{y} = f_L(...f_2(f_1(\boldsymbol{x})))$$

During the Forward Pass, each layer performs an Affine Transformation (a linear rotation and scaling of data via weight matrix $\boldsymbol{W}$ and bias $\boldsymbol{b}$) followed by a Non-Linear Activation (σ):

$$\boldsymbol{z}^{(l)} = \boldsymbol{W}^{(l)} \boldsymbol{a}^{(l-1)} + \boldsymbol{b}^{(l)}$$

$$\boldsymbol{a}^{(l)} = \sigma\left(\boldsymbol{z}^{(l)}\right)$$

The non-linearity prevents the deep stack from collapsing into a single linear equation. Modern networks rely on the Rectified Linear Unit (ReLU), $f(x) = \max(0, x)$. Its derivative (0 or 1) preserves the magnitude of the gradient, allowing error signals to travel through deep structures without vanishing.

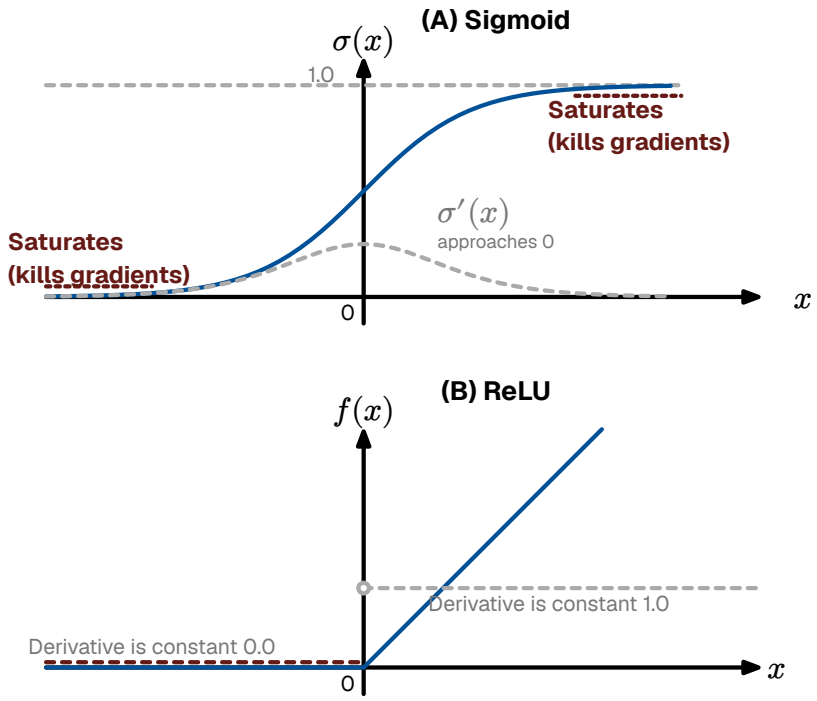

Figure 18: Activation Functions. The Sigmoid (left) saturates gradients. The ReLU (right) preserves gradient magnitude for positive inputs.

During the Backward Pass, Backpropagation recursively applies the Chain Rule via Automatic Differentiation to attribute the total error $J(\boldsymbol{\theta})$ to specific weights.

To achieve high throughput, these operations are vectorized. The affine transformation for an entire layer is executed as a Dense Matrix Multiplication. This is the defining characteristic of modern AI hardware. A deep network is effectively a sequence of massive matrix multiplications, which is highly parallelizable on GPUs

$$\begin{bmatrix} w & w & w & w \end{bmatrix} \begin{bmatrix} x \\ x \\ x \\ x \end{bmatrix} \quad \begin{bmatrix} w & w & w & w \\ w & w & w & w \\ w & w & w & w \\ w & w & w & w \\ w & w & w & w \\ w & w & w & w \end{bmatrix} \begin{bmatrix} x \\ x \\ x \\ x \end{bmatrix}$$

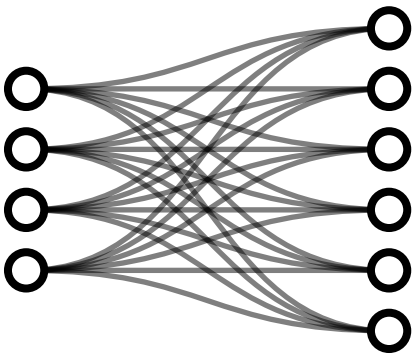

Figure 19: Deep Learning as Matrix Multiplication. Forward and backward passes rely on dense matrix products, necessitating high-bandwidth memory access.

4.2.1 • CONVOLUTIONAL NEURAL NETWORKS (CNNS)

For visual tasks, standard Multi-Layer Perceptrons scale poorly; connecting every pixel to every neuron ignores the spatial structure of the data and creates an intractable number of weights. To solve this, DL utilizes CNNs.

CNNs apply small, learnable weight matrices known as "kernels" or "filters" that slide (convolve) across the input image. This architecture introduces two critical inductive biases:

Local Connectivity: Neurons only process a small, local receptive field, analogous to the biological visual cortex.

Weight Sharing: The exact same kernel is applied across the entire image, drastically reducing the number of tunable parameters and establishing translation invariance (a feature learned in one corner of an image can be recognized anywhere else).

While CNNs are the standard baseline for spatial processing, they remain fundamentally synchronous and frame-based, evaluating the entire image structure in dense mathematical passes regardless of local activity.

4.3 • WHY IS DEEP LEARNING INEFFICIENT?

While the matrix-centric formulation of Deep Learning enables high-throughput parallelization on GPUs, it fundamentally conflicts with the physical constraints of modern computing hardware. As models scale to billions of parameters, the primary bottleneck shifts from algorithmic capability to physical realizability. This inefficiency manifests in four distinct engineering dimensions:

4.3.1 • THE VON NEUMANN BOTTLENECK & DATA MOVEMENT

The most significant physical limitation is the Von Neumann Architecture, which physically separates the Processing Unit from the Memory Unit. To perform a single inference step, the processor must fetch the entire weight matrix from off-chip DRAM to on-chip registers, perform the calculation, and write the results back.

According to Horowitz and Dally [23], fetching a 32-bit value from off-chip DRAM consumes approximately 640 pJ, whereas performing a floating-point addition on that value consumes only 0.1 pJ. The system expends 99.9% of its energy transporting data, and only 0.1% actually computing.

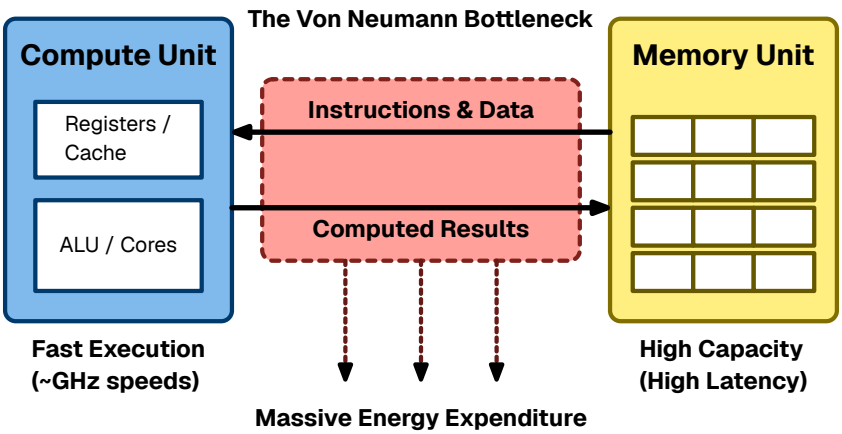

Figure 20: The Von Neumann Bottleneck. The separation of memory and compute forces massive energy expenditure on data transport.

4.3.2 • DENSE PROCESSING OF SPARSE DATA

Standard Deep Learning implementations rely on Dense Matrix Multiplication (GEMM). This approach is algorithmically rigid: it executes the same number of operations regardless of the data content.

Real-world sensory data is often highly sparse, and the ReLU activation function naturally produces activation maps where the majority of values are zero. However, a standard GPU is "blind" to this sparsity. It will dutifully fetch a zero from memory and multiply it by a weight ($0 \times w = 0$), consuming energy and clock cycles to produce a null result. Deep Learning's inability to exploit this silence represents a massive structural inefficiency.

4.3.3 • THE HIGH COST OF SYNCHRONY

Deep Learning hardware is typically Synchronous, operating in lockstep with a global clock. Driving a high-frequency clock signal across an entire silicon die forces billions of transistors to charge and discharge continuously, regardless of whether the chip is doing useful work. In high-performance processors, this clock distribution network alone can consume 30% to 40% of the total power budget. Furthermore, global synchronization enforces a "worst-case" latency: faster computations must sit idle and wait for the slowest operations to finish before the next clock cycle begins.

4.3.4 • BACKPROPAGATION AND GLOBAL DEPENDENCIES

Finally, Backpropagation imposes severe constraints on memory and latency because it is non-local in both time and space. To update a specific weight, the system must wait for the Forward Pass to finish, calculate the global error, and wait for the backward pass to propagate the gradient.

This creates a "Locking Problem." The activations of every intermediate layer must be stored in high-speed memory (VRAM) for the duration of the entire pass, prevent-

ing that memory from being reused. Additionally, a local synapse cannot adapt to local changes instantly; it is shackled to the global error loop.

4.4 • PRINCIPLES OF NEUROMORPHIC ENGINEERING

As established in the *History & Developments* chapter, Neuromorphic Engineering is the translation of biological dynamics into silicon hardware. It replaces the rigid, clock-driven logic of standard computing with the adaptive, event-driven dynamics of neural tissue. This approach rests on three architectural pillars that directly address the bottlenecks of Deep Learning:

Co-location of Memory and Compute (The Synaptic Principle): Neuromorphic architectures eliminate the Von Neumann bottleneck by distributing memory across the silicon die. Each artificial neuron stores its own state and synaptic weights locally, processing data **in situ** to eliminate the energy cost of shuttling data.

Event-Driven Asynchrony (The Action Potential Principle): Neuromorphic systems abandon the global clock. They operate asynchronously, driven strictly by the arrival of data. If a part of the network is not processing information, it consumes negligible power, ensuring energy scales linearly with task complexity rather than network size.

Sparse Communication (The Spike Principle): Neuromorphic systems utilize binary Spikes for communication. Information is encoded in the precise timing of events rather than complex magnitudes, drastically reducing the bandwidth required to transmit information between neurons.

4.5 • TRAINING SPIKING NETWORKS

While the physical architecture of neuromorphic systems is highly efficient, training these networks presents a fundamental mathematical challenge. Standard deep learning relies on gradient descent, but backpropagation cannot be directly applied to native SNNs.

In a spiking network, the neuron's activation function is a discontinuous step function (the Dirac delta event threshold). The derivative of this function is zero everywhere except at the exact moment of the spike, where it is undefined. Consequently, gradients calculated using the chain rule immediately drop to zero—known as the "Dead Neuron" problem—preventing error signals from flowing backward through the network to update the weights.

To circumvent this non-differentiability and optimize network parameters, the field of neuromorphic engineering generally employs two distinct paradigms:

4.5.1 • DIRECT WEIGHT TRANSFER

A pragmatic engineering approach to bypass the dead neuron problem is offline training. In this paradigm, a standard, continuous ANN (such as a network utilizing ReLU activations) is trained conventionally using backpropagation. Once convergence is achieved, the learned weights are directly mapped onto a structurally identical Spiking Neural Network.

The underlying premise is that the continuous activation values of the ANN can be approximated by the discrete firing rates of the SNN over a set time window. While this method allows the spiking system to inherit the high accuracy of gradient descent, direct weight transfer requires careful scaling and normalization. If the weights are copied without adjustment, the resulting SNN may suffer from catastrophic saturation (firing constantly) or severe signal degradation (failing to reach the spiking threshold). This conversion process and its associated quantization constraints are evaluated in our implementation in Section 6.4.1.

4.5.2 • NATIVE LOCAL LEARNING (STDP)

To fully exploit the energy efficiency and event-driven dynamics of neuromorphic hardware, training must ideally occur natively on the spiking substrate. This requires abandoning global backpropagation in favor of biologically plausible, mathematically local learning rules.

As established in Section 3.6, Spike-Timing-Dependent Plasticity (STDP) adjusts synaptic weights based strictly on the temporal correlation of local pre- and post-synaptic spikes. Because STDP relies exclusively on local physical events rather than global error gradients, it does not require a differentiable loss function. This allows the network to completely bypass the dead neuron problem, enabling unsupervised feature extraction and real-time adaptation directly on the spiking architecture.

4.5.3 • SURROGATE GRADIENT DESCENT

For completeness, it must be noted that the current dominant paradigm in SNN research utilizes Surrogate Gradients. In this approach, the network operates using the discontinuous spike step-function during the forward pass, but temporarily replaces the undefined derivative with a smooth, continuous approximation (a “surrogate”) during the backward pass. While this thesis focuses on evaluating direct weight transfer and native unsupervised STDP, surrogate methods represent a highly effective hybrid approach, allowing backpropagation-like algorithms to estimate gradients across discrete spiking layers.

4.6 • NEUROMORPHIC HARDWARE TECHNIQUES

Central to realizing these computational efficiencies in physical hardware is Adress Event Representation (AER), a communication protocol that mirrors the sparse nature of biological spikes. Instead of continuous data streaming, the hardware only transmits the "address" of a firing neuron across a shared digital bus, allowing a single physical wire to represent thousands of virtual axonal projections.

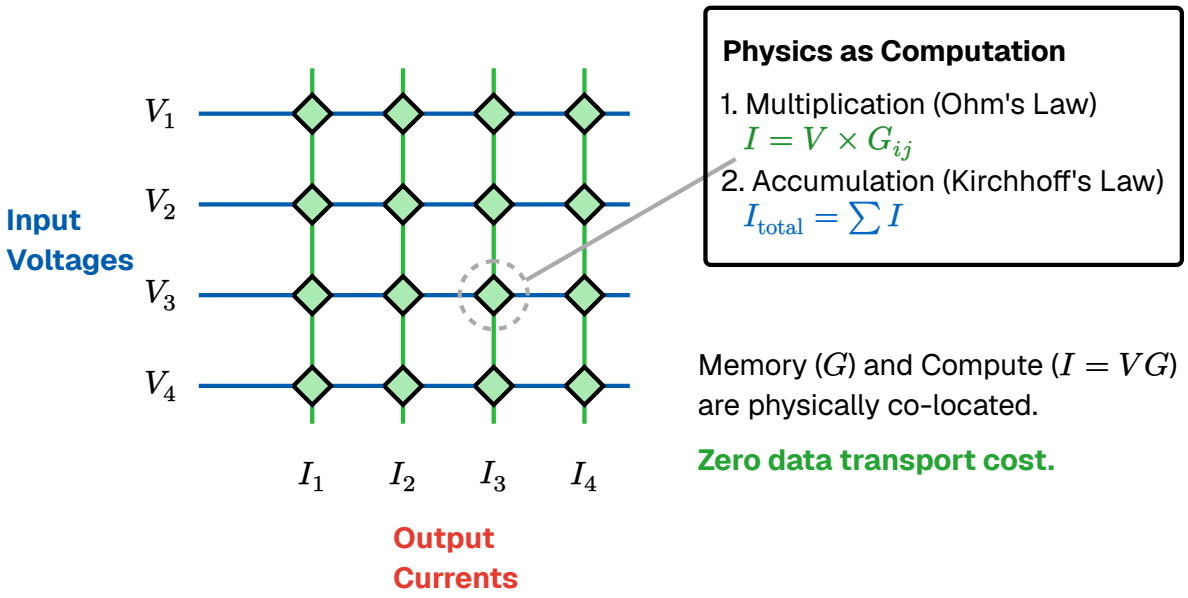

Figure 21: In-Memory Computing via a Crossbar Array. Unlike von Neumann architectures, memory and computation are physically co-located. Input voltages (V) are applied to the wordlines. Memory elements at the junctions hold programmable conductances (G). Multiplication is natively performed at each junction by Ohm's Law ($I = V \times G$), and resulting currents are summed along the bitlines via Kirchhoff's Current Law. This allows dense matrix-vector multiplications to occur in a single analog time step with zero data transport cost.

The crossbar array provides a direct structural surrogate for the neural neuropil. Because the architecture handles multiplication and summation natively through physical laws, it is uniquely suited to implement biological "macro-motifs." By routing bitline currents through local feedback loops, the hardware can instantiate complex dynamics such as Lateral Inhibition and Winner-Take-All circuits without the overhead of high-level software instructions.

This synergy between physical topology and functional motifs allows the hardware to inherit the computational efficiency of the neocortex, effectively making the architecture itself the algorithm.

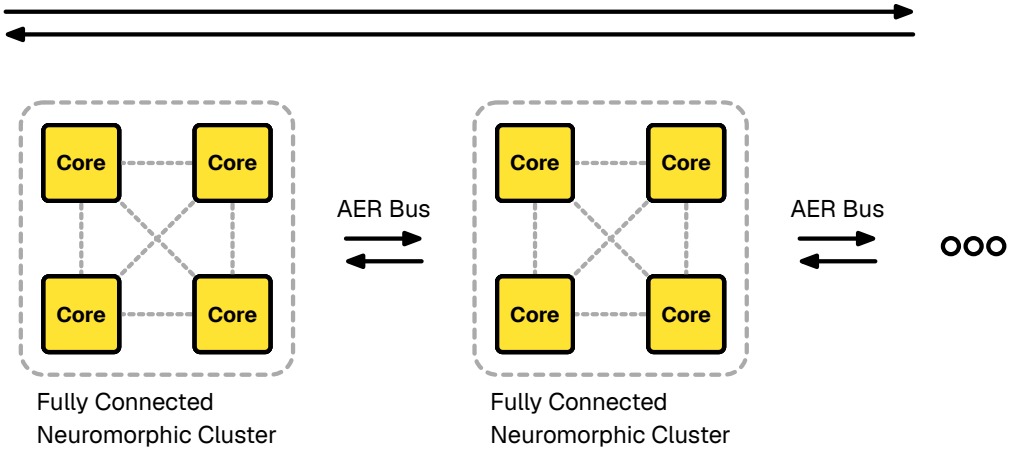

Figure 22: Architectural Comparison. (Left) The Von Neumann architecture separates memory and compute, creating a bottleneck. (Right) The Neuromorphic architecture co-locates them, mimicking the distributed topology of biological neural networks.

5 • RELATED WORKS

The shift towards neuromorphic computing is largely driven by the urgent need for energy-efficient machine intelligence. Recent literature highlights a rapidly expanding landscape of opportunities for neuromorphic algorithms, moving beyond isolated benchmarks toward real-world applications. The methodologies implemented in this thesis—specifically temporal coding, sparsity, and local learning—intersect directly with several active areas of state-of-the-art research.

5.0.1 • TIME-BASED CODING AND BIOLOGICAL PLAUSIBILITY

The foundational premise of our TTFS architecture is that precise spike timing carries significant information. This is powerfully supported by recent biological evidence; in vivo recordings from the human cortex confirm that neuronal sequences during population bursts explicitly encode information through their temporal order [36].

Translating these biological temporal sequences into functional artificial networks has been a major focus of recent engineering efforts. Han and Roy have demonstrated that deep spiking neural networks can achieve extreme energy efficiency specifically by leveraging time-based coding paradigms [37]. Concurrently, theorists are continually refining how biological constraints—such as excitation-inhibition balance and adaptive homeostatic currents—can be integrated into leaky integrate-and-fire networks to achieve optimal efficient coding.

5.0.2 • HARDWARE ACCELERATION AND SPARSITY

A core theoretical advantage of the TTFS encoding utilized in our experiments is spatial and temporal sparsity. However, realizing these efficiency gains requires bridging the gap between software simulation and physical deployment. Recent SOTA implementations have focused heavily on exploiting this sparsity directly on customized hardware. For example, Sommer et al. [38] proposed novel memory interlacing and hardware acceleration architectures for sparsely active convolutional SNNs on Field Programmable Gate Arrays (FPGAs), achieving significant speedups by ensuring processing time scales directly with the number of spikes.

Furthermore, event-driven algorithms are increasingly being paired with native event-based sensors. Applications such as spiking optical flow estimation from dynamic vision sensors have been successfully deployed on specialized neuromorphic substrates like IBM's TrueNorth [39], proving the viability of low-power, asynchronous processing in dynamic environments.

5.0.3 • ANN-TO-SNN CONVERSION

A significant portion of contemporary neuromorphic engineering focuses on bridging the gap between classical Deep Learning and spiking hardware via offline training. Foundational work by Rueckauer et al. (2017) demonstrated that continuous-valued networks could be losslessly converted into SNNs by carefully scaling weights and normalizing spiking thresholds `@rueckauer_conversion_2017`. However, the vast majority of these conversion methodologies rely on Rate Coding, requiring hundreds of simulation ticks to approximate the continuous activation values. Applying zero-shot weight transfer directly to a Time-To-First-Spike (TTFS) encoding, as explored in Phase II of this thesis, represents a distinct and more sensitive challenge, as quantization errors directly impact temporal priority rather than just average frequency.

5.0.4 • ADVANCEMENTS IN SEQUENCE MODELING

While the experimental scope of this thesis utilizes static visual classification (MNIST) to isolate temporal integration dynamics, the broader SNN field is actively expanding into complex, long-term sequential tasks. Recent breakthroughs have investigated the intersection of SNNs with modern State Space Models [40]. This research demonstrates that spiking architectures can now successfully compete with —and in terms of parameter efficiency, outperform—traditional Transformer models on long-range sequence modeling tasks.

5.0.5 • UNSUPERVISED STDP AND COMPETITIVE LEARNING

The application of unsupervised Spike-Timing-Dependent Plasticity (STDP) to benchmark datasets like MNIST serves as the standard baseline for evaluating biologically plausible learning. The foundational architecture for this approach was established by Diehl and Cook (2015), who demonstrated that a spiking network utilizing STDP, lateral inhibition, and adaptive homeostatic thresholds could achieve 95% accuracy on MNIST without any labels `@diehl_unsupervised_2015`.

However, it is critical to note that their state-of-the-art result relied on a massive spatial population (6,400 hidden neurons) and extended rate-based integration windows to form robust overlapping receptive fields. Evaluating these same STDP and homeostatic principles under the severe constraints of a small, 128-neuron architecture utilizing strict TTFS single-spike encoding, as implemented in Phase III of this thesis, provides unique insights into the limits of temporal feature clustering when spatial redundancy is removed.

6 • METHOD

This chapter details the specific implementations of the neuromorphic architectures proposed to address the limitations of standard deep learning. Aligning with the biological constraints of sparsity, asynchrony, and locality established in previous chapters, we outline the construction and evaluation of a SNN.

To empirically validate the theoretical advantages of neuromorphic algorithms, we evaluate the system on a benchmark image classification task. The experiment is bifurcated into three distinct phases:

Phase I—Neuron Model Evaluation: Evaluating the decoding efficiency and accuracy of different simulated spiking models.

Phase II—Inference Via Weight Transfer: Evaluating the zero-shot performance of these SNNs initialized with weights directly mapped from a classically trained Artificial Neural Network (ANN).

Phase III—Native Unsupervised Learning: Training the SNN from scratch utilizing local STDP.

6.1 • DATASET & PRE-PROCESSING

To benchmark these algorithms, we require a dataset that necessitates the extraction of complex spatial features but remains computationally tractable for rapid experimental iteration. We utilize the MNIST database of handwritten digits [41].

The dataset consists of a training set of 60,000 examples and a test set of 10,000 examples of digits (0-9). Each instance is a 28×28 pixel grayscale image. While standard deep learning models routinely score over 90% accuracy on this task, making it largely a solved problem in classical AI, its well-understood feature space makes it an ideal, isolated baseline. Because the spatial hierarchy of MNIST is relatively shallow, it allows us to evaluate the efficacy of neuromorphic learning rules without the confounding variables introduced by massive, multi-layered convolutional architectures.

Crucially, the MNIST images are pre-processed by the dataset creators to be size-normalized and centered within the pixel grid using the center of mass of the pixels. This spatial alignment is a vital prerequisite for our chosen network topology. Unlike CNNs, which slide localized filters across an image, the FCN utilized in this thesis lacks translation invariance. If a digit were shifted several pixels off-center, the FCN would perceive it as an entirely novel pattern. The pre-centered nature of MNIST mitigates this limitation, ensuring that the network can reliably map specific geometric strokes to specific input neurons.

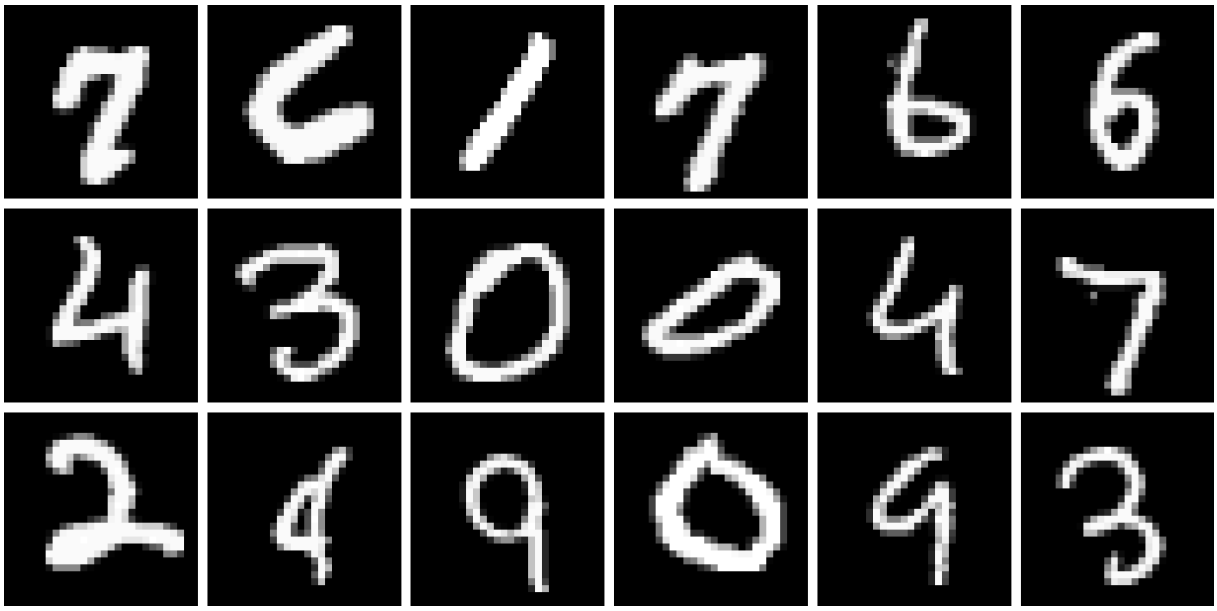

Figure 23: Sample of the MNIST dataset. The 28x28 images are normalized and flattened into 1D vectors before being translated into temporal spike events.

Furthermore, the dataset exhibits a high degree of spatial sparsity. In a typical MNIST image, the vast majority of pixels represent the empty background. From a neuromorphic engineering perspective, this sparsity is highly advantageous. As established in the theoretical framework, event-driven systems expend energy strictly when events occur. A sparse input array ensures that the majority of input neurons remain quiescent, minimizing bus congestion and validating the energy-efficiency claims of the proposed SNN.

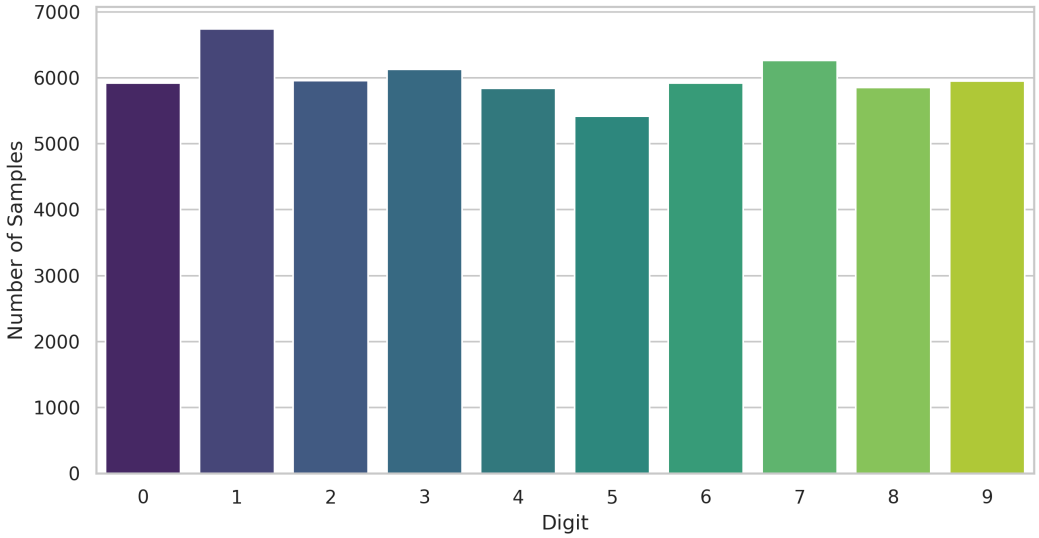

Figure 24: Sample of the MNIST dataset. The 28x28 images are normalized and flattened into 1D vectors before being translated into temporal spike events.

Before the raw images can be converted into temporal spike trains, they must undergo standard spatial pre-processing to ensure compatibility with the network's mathematical boundaries. This consists of two primary transformations:

Normalization: Raw pixel intensities in the MNIST dataset range from 0 (pure black) to 255 (pure white). To stabilize the learning algorithms and ensure consistent weight scaling, these values are strictly normalized to a continuous float range of $p_i \in [0.0, 1.0]$.

Flattening: Because this thesis utilizes a Fully Connected Network (FCN) to facilitate direct weight transfer, the 2D spatial structure of the images must be unrolled. Each 28×28 matrix is flattened into a 1-dimensional vector of 784 elements.

Consequently, every individual image is presented to the system as a discrete array of 784 normalized intensities. In the classical Artificial Neural Network (ANN), these continuous values are fed directly into the input neurons. However, because SNNs operate exclusively on discrete events, these normalized values must be passed through a temporal encoding algorithm before inference or learning can begin.

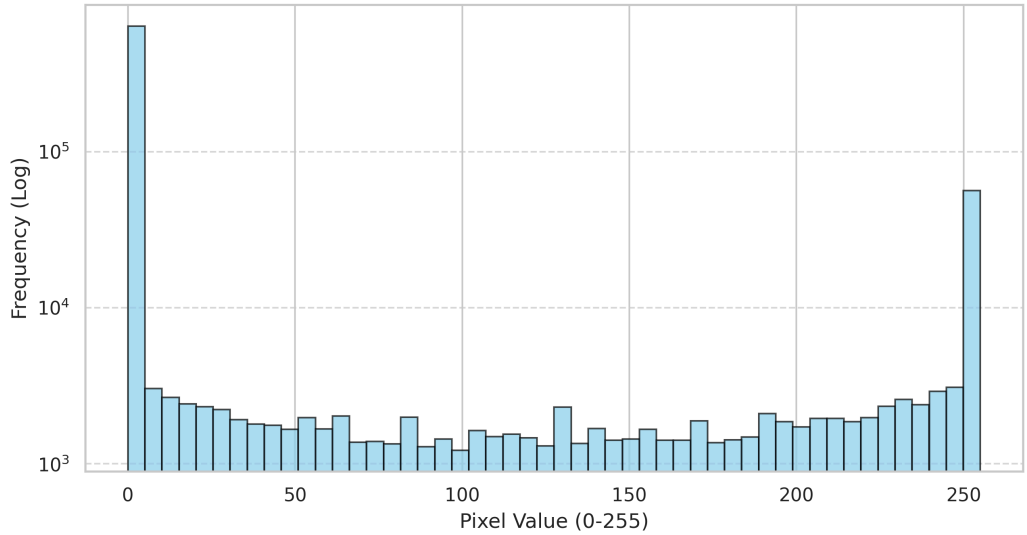

Figure 25: Sample of the MNIST dataset. The 28x28 images are normalized and flattened into 1D vectors before being translated into temporal spike events.

6.2 • NETWORK ARCHITECTURE

To facilitate a direct, one-to-one mapping of synaptic weights from the ANN to the SNN, both models must share an identical macroscopic topology. That is the motivation for why this implementation utilizes a FCN, also known as a MLP, rather than a CNN.

While CNNs are the standard baseline for vision tasks due to their spatial inductive biases, transferring convolutional kernels into a spiking substrate introduces signif-

icant mapping complexities—specifically the need to physically unroll and duplicate shared weights across the spiking array. An FCN provides a straightforward, mathematically transparent architecture for cleanly evaluating direct weight transfer and STDP without confounding architectural variables.

The network is structured as a shallow hierarchy to capture the primitive geometric features of the dataset. Let N_l denote the number of neurons in layer l. The formal architecture is defined as follows:

Input Layer (L_0): The 28×28 pixel grayscale images are flattened into a 1D vector, requiring $N_0 = 784$ input neurons.

Hidden Layer (L_1): A fully connected intermediate layer consisting of $N_1 = 128$ neurons. The synaptic connections are defined by the weight matrix $W^{(1)} \in \mathbb{R}^{\{N_1 \times N_0\}}$.

Output Layer (L_2): $N_2 = 10$ neurons, corresponding directly to the categorical digit classes (0 through 9). The connections from the hidden layer are defined by the weight matrix $W^{(2)} \in \mathbb{R}^{\{N_2 \times N_1\}}$.

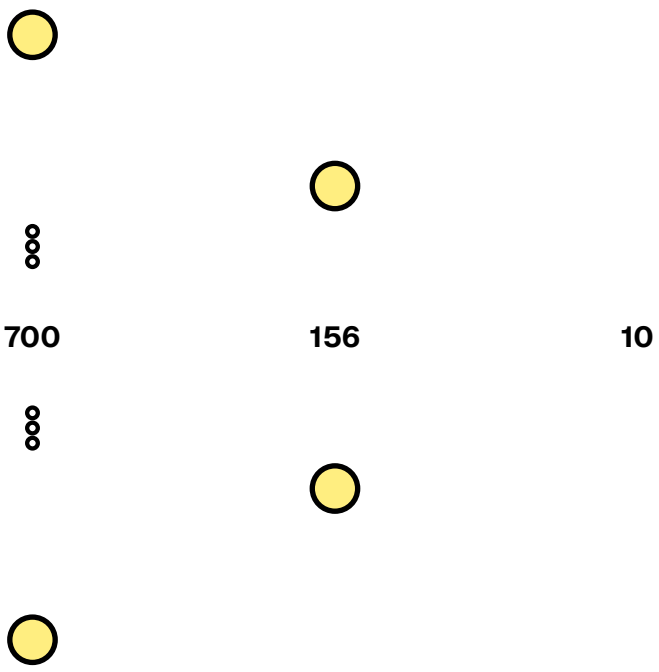

Figure 26: Network architecture. The 28x28 images are flattened and passed through a Fully Connected Network. Both the Artificial Neural Network (ANN) and Spiking Neural Network (SNN) share this identical macroscopic topology.

6.2.1 • LATERAL INHIBITION AND WTA

Lateral inhibition and WTA is unique to the SNN and is a way to enforce competitive learning and strict decision making. To implement the WTA dynamics established in Section 3.5.4, the network heavily relies on recurrent inhibitory connections. In the spiking simulation, a generic inhibitory blast of $w_{\text{lat}} = -200.0$ is distributed to suppress competing hidden neurons. Furthermore, when evaluating the output clas-

sification layer, the first neuron to cross its threshold triggers a strict WTA condition, instantly suppressing all competitors to finalize the classification and halting further integration.

6.2.2 • WEIGHT INITIALIZATION AND BIASES

For the baseline ANN, the weight matrices are initialized using standard PyTorch defaults to ensure stable variance during the initial forward passes.

A critical architectural consideration in ANN-to-SNN conversion is the handling of bias vectors ($b^{(l)}$). While standard ANNs utilize biases to shift activation functions, representing a static continuous bias in a spiking network requires either injecting a constant background current into the neuron or actively modifying the neuron's firing threshold. To maintain pure, event-driven sparsity and isolate the performance of the synaptic weights, explicit bias terms were omitted entirely from both architectures.

6.2.3 • ANN-SNN COMPATIBILITY

Despite the macroscopic symmetry required for weight sharing, the microscopic dynamics of the two networks differ fundamentally. The offline ANN utilizes standard continuous activation functions (specifically ReLU) to compute smooth gradients during backpropagation.

In contrast, the SNN replaces these continuous functions with Integrate-and-Fire neurons governed by a strict voltage threshold. This simulates the biological "all-or-nothing" action potential, acting as a hard step function. Furthermore, the spiking architecture utilizes lateral inhibition at the output layer. This engenders a WTA dynamic: as the network integrates evidence over time, the first output neuron to reach its threshold heavily suppresses its competitors, forcing a definitive categorical decision and actively filtering sub-threshold noise.

6.3 • SNN INFORMATION REPRESENTATION

The choice of neural code lays the foundation for information flow and dictates the efficiency of the entire system. While Rate Coding (encoding pixel intensity as spike frequency) is straightforward and simple to implement with standard Integrate-and-Fire neurons, it is inefficient compared to TTFS. Rate codes require integration over extended time windows to calculate an average, introducing latency and saturating the network bus with redundant spikes. Furthermore, on digital hardware rate coding imposes additional stress on the system due to rapid switching which is very bad for transistor power draw and bus congestion.

To maximize energy efficiency and processing speed, this implementation utilizes a TTFS temporal encoding [42]. In this regime, a single spike carries the information

payload. A high-intensity (bright) pixel triggers an early spike, while a low-intensity (dark) pixel triggers a late spike. This compresses the spatial information into a highly sparse, priority-driven queue; downstream neurons begin processing as soon as the most salient features arrive, without waiting for an entire frame to integrate.

As noted in Section 3.4, temporal codes suffer from Phase Ambiguity—downstream neurons need a reference "clock" to decode latency. To resolve this without relying on a rigid, global system clock, we simulate the biological concept of a saccade (the rapid movement of the eye to fixate on a target). The initial presentation of the image acts as a synchronized global event (t_0). All subsequent input spikes are evaluated relative to this saccade onset, providing a natural, biologically plausible temporal reference frame.

6.3.1 • ENCODING

Following the TTFS principles described in Section 3.4.2, we convert the continuous pixel intensities of the MNIST dataset into discrete spike trains. For a given input image, we extract the luminance of each pixel and normalize it to a bounded range, where $p_i \in [0, 1]$. 1 representing maximum intensity and 0 representing the background).

We implement a single, highly sparse linear conversion mapping to evaluate latency dynamics. The pixel intensity $p_i \in [0.0, 1.0]$ is inverted such that brighter pixels correspond to shorter delays. In our implementation, the maximum delay permitted for a valid pixel is 32 ticks, despite the full saccade window being 64 ticks. Furthermore, to aggressively enforce input sparsity, any pixel with an intensity below 0.1 is treated as background noise and discarded (assigned a delay of infinity, meaning it never fires).

$$t_i = \begin{cases} \text{round}((1.0 - p_i) \cdot 32) & \text{if } \; p_i \geq 0.1 \\ \infty & \text{if } \; p_i < 0.1 \end{cases}$$

Equation 9: Intensity-to-Delay Encoding Implementation

Under this mapping, the brightest pixels fire immediately near $t = 0$, transmitting the most critical structural features of the digit first, while background pixels are entirely suppressed.

6.3.2 • DECODING WITH NEURON MODELS

Decoding the temporal information generated by the TTFS encoding requires a neuron model capable of accumulating discrete events. However, a fundamental engineering tension exists between biological fidelity, computational efficiency, and how a model dynamically reacts to temporal density.

To systematically evaluate these trade-offs, this thesis implements and benchmarks four distinct neuron models. These models range from simple arithmetic accumulators requiring global synchronization to complex, fully asynchronous dynamic

systems. By adjusting the threshold bounds during Phase I experiments, we evaluate how each distinct mathematical approach processes incoming spikes and updates its membrane potential $V_{m(t)}$ when an event arrives at time t, carrying a synaptic weight w_i.

MODEL A: THE SIMPLE WINDOW INTEGRATOR (STANDARD IF)

The most computationally lightweight approach is the standard Integrate-and-Fire (IF) model without any leak or decay mechanisms. In this paradigm, the neuron acts as a pure arithmetic accumulator during the simulation window.

$$u(t_i^+) = u(t_{i-1}^+) + w_i$$

Equation 10: Discrete Recurrence Relation for Model A

Computational Complexity: Minimal. This model is extremely cheap to execute on digital hardware, requiring only a single addition operation $O(1)$ per incoming spike.

Temporal Dynamics: Static. While highly efficient, this model's integration is purely additive. Its temporal dynamics are rigidly tied to threshold calibration; if the threshold is low, it fires prematurely based solely on early magnitude accumulation, remaining blind to the wider temporal distribution of the spike train.

Synchronization: Because the potential never naturally decays, this model relies entirely on a rigid, globally synchronized saccade (a hard reset of u to 0 at the end of the simulation window) to prevent the network from firing continuously due to lingering historical noise.

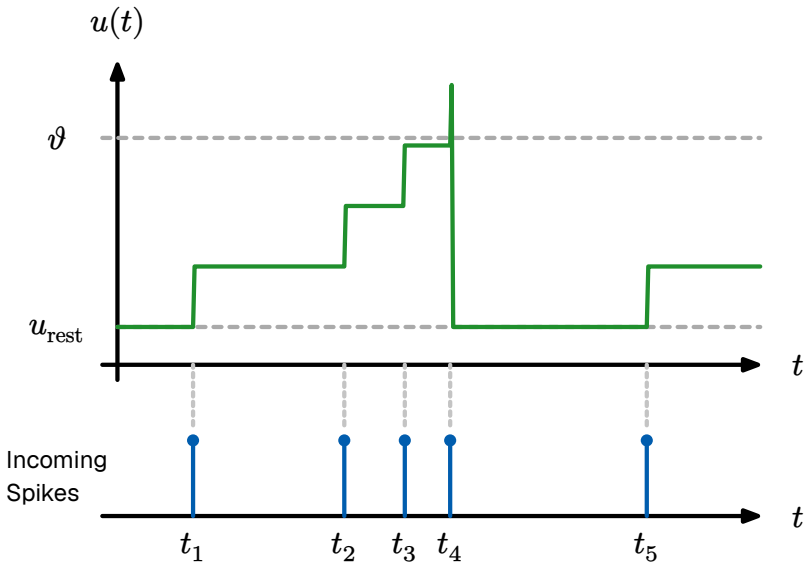

Figure 27: Network architecture. The 28x28 images are flattened and passed through a Fully Connected Network. Both the Artificial Neural Network (ANN) and Spiking Neural Network (SNN) share this identical macroscopic topology.

```
simpleIntegratorNeuron(S, W, theta) → t_fire:
    u ← 0.0
    for each (t_i, w_i) ∈ S:
        u ← u + w_i
        if u ≥ θ:
            return t_i
    return ∞
```

Algorithm 1: Model A: Simple Window Integrator Algorithm

MODEL B: THE STANDARD LEAKY INTEGRATE-AND-FIRE (LIF)

Model B—the LIF model is covered in great detail in Section 3.3.1. Model B fires only if the incoming spike train has a sufficient density of spikes. A sparse spike train cannot overcome the exponentially decaying membrane potential.

$$u(t_i^+) = \max\left(0, u(t_{i-1}^+) \cdot \exp\left(-\frac{t_i - t_{i-1}}{\tau_m}\right) + w_i\right)$$

Equation 11: Discrete Recurrence Relation for Model B

Computational Complexity: High. This model is significantly more intensive, requiring the calculation of exponential functions for every discrete event, which consumes substantial clock cycles on standard arithmetic logic units.

Temporal Dynamics: Decay-driven. The exponential leak provides a basic temporal filter that naturally favors spikes arriving in rapid succession. However, its effectiveness is highly dependent on both threshold constraints and the time constant τ_m. If the threshold is set too high relative to the input density, the signal degrades before threshold crossing, preventing firing entirely.

Synchronization: Similar to Model A, while the leak reduces residual noise, it generally still requires a global saccade reset between distinct inference phases to guarantee a clean slate for the next image.

```
leakyIntegratorNeuron(S, W, theta, tau_m) → t_fire:
    u ← 0.0
    t_prev ← 0.0
    for each (t_i, w_i) ∈ S:
        Δt ← t_i − t_prev
        u ← max(0.0, u · exp(−Δ t/τ_m) + w_i)
        if u ≥ θ:
            return t_i
        t_prev ← t_i
    return ∞
```

Algorithm 2: Model B: Standard Leaky Integrate-and-Fire Algorithm

MODEL C: THE CURRENT-ACCUMULATING LINEAR RAMP

To explore dynamic accumulation without the computational overhead of continuous exponential kernels, Model C utilizes a linear time-dependent accumulator combined with a strict hard reset timer (coincidence_window = 10 ticks). The arrival of a spike increments both the instantaneous potential $u(t)$ and the integration gradient $I(t)$. However, if the neuron does not reach the threshold within 10 ticks of the initial spike, the timer expires and both state variables are aggressively wiped to 0.0. This is achieved by modeling the membrane potential $u(t)$ as a system of coupled differential equations driven by a sequence of Dirac delta impulses:

$$\dot{I}(t) = \sum_i w_i \delta(t - t_i)$$

$$\dot{u}(t) = I(t) + \sum_i w_i \delta(t - t_i)$$

This formulation ensures that an afferent spike at time t_i with weight w_i exerts a dual influence: it induces an immediate translocation of the potential state while simultaneously incrementing the rate of change for all subsequent integration intervals. Because early spikes establish the baseline slope $I(t)$ for the remainder of the simulation window, they exert a disproportionate momentum on the time-to-threshold.

$$I(t_i^+) = I(t_{i-1}^+) + w_i$$

$$u(t_i^+) = u(t_{i-1}^+) + I(t_{i-1}^+) \cdot (t_i - t_{i-1}) + w_i$$

Equation 12: Discrete Recurrence Relations for Model C

Computational Complexity: Optimized. The model replaces power-intensive transcendental operations with floating-point additions and a single multiplication per spike event ($I \cdot \Delta t$).

Temporal Dynamics: Highly reactive. The quadratic-like growth of the potential relative to spike arrival times means earlier spikes violently accelerate the trajectory toward the threshold. By shifting the threshold bounds, we can observe how this momentum-driven integration drastically alters firing latency compared to standard static accumulators.

Synchronization: This model requires a rigorous global synchronization protocol (e.g., a saccade-driven clock). In the absence of a periodic global reset to zero the state variables $I(t)$ and $u(t)$, the linear integration would diverge toward hardware saturation limits.

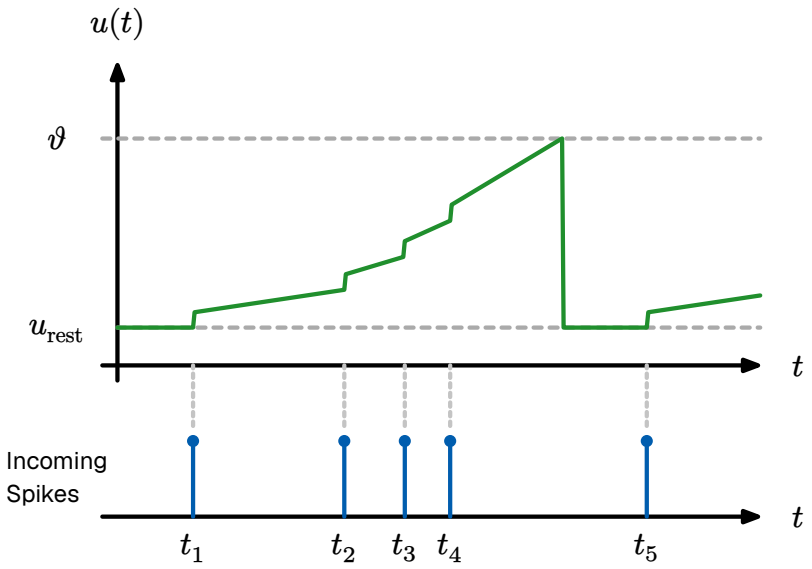

Figure 28: Evolution of state variables in Model C. This demonstrates the resulting piecewise linear membrane potential $u(t)$. Note how earlier spikes increase the integration gradient, accelerating the trajectory toward the firing threshold θ.

```
linearRampNeuron(S, W, theta) → t_fire:
    u ← 0.0
    I ← 0.0
    t_prev ← 0.0
    for each (t_i, w_i) ∈ S:
        Δt ← t_i − t_prev
        u ← u + (I · Δt) + w_i
        I ← I + w_i
        if u ≥ θ:
            return t_i
        t_prev ← t_i
    return ∞
```

Algorithm 3: Model C: Discrete State Update Algorithm

MODEL D: THRESHOLD-SENSITIVE INTEGRATION (STATE-DEPENDENT DISCOUNTING)

Drawing inspiration from the adaptation mechanisms of the GLIF model (Section 3.3.2), Model D introduces a state-dependent penalty to incoming spikes. Rather than penalizing spikes based on the passage of time, this model penalizes spikes based on the current internal state of the neuron.

In this paradigm, the increase in membrane potential is inversely proportional to the current potential itself. When a spike arrives at a neuron in its resting state ($u = 0$), there is no discount, and the full synaptic weight is added. However, if a spike arrives when the neuron is already close to the firing threshold θ, its impact is exponentially discounted.

$$u(t_i^+) = u(t_{i-1}^+) + w_i \cdot \exp\left(-\gamma \frac{u(t_{i-1}^+)}{\theta}\right)$$

Equation 13: Discrete Recurrence Relation for Model D Weight Discounting

Computational Complexity: Moderate. While it requires an exponential calculation (or a linear approximation), it does not need to continuously track the elapsed time (Δt) between individual spikes, saving memory overhead compared to continuous-time dynamic models.

Temporal Dynamics: State-dependent. This mechanism creates a unique temporal compression. The earliest spikes contribute their absolute maximum weight, while late arrivals encounter a partially filled potential and are severely dampened. Testing this model across varying thresholds reveals an asymptotic behavior where the neuron may never fire if the initial temporal momentum is insufficient to overcome the compounding discount.

Synchronization: Similar to Models A and C, this model requires a rigorous global synchronization protocol (e.g., a saccade-driven clock). Because this model drops the continuous temporal leak in favor of event-driven weight scaling, the potential does not naturally decay to zero between images and relies on a periodic reset at the conclusion of the simulation window.

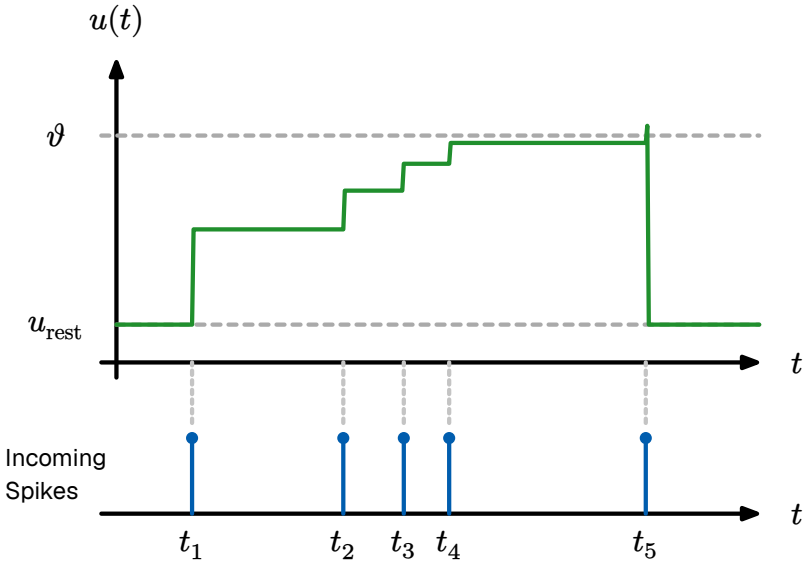

Figure 29: Evolution of state variables in Model D. This demonstrates a neuron model where new incoming spikes have an exponentially decaying influence based on the current potential $u(t)$.

```
thresholdSensitiveNeuron(S, W, theta, gamma) → t_fire:
    u ← 0.0
    for each (t_i, w_i) ∈ S:
        η ← exp(−γ · u/θ)
        u ← u + (w_i · η)
        if u ≥ θ:
            return t_i
    return ∞
```

Algorithm 4: Model D: Threshold-Sensitive Integration Algorithm

THRESHOLDING

To rigorously evaluate these distinct temporal dynamics, Phase I of our methodology introduces a variable thresholding benchmark. Rather than relying on a single fixed threshold to measure performance, we systematically adjust the membrane threshold (θ) relative to the total sum of the incoming synaptic weights ($\sum w_i$).

If θ is configured significantly lower than $\sum w_i$, models naturally reach early saturation, firing prematurely based strictly on the magnitude of early-arriving spikes. By sweeping the threshold from low to high (approaching the total weight sum), we can empirically demonstrate how the different internal mechanics—perfect integration, leaky decay, momentum accumulation, and state-discounting—respond to the exact same temporal patterns under varying saturation constraints.

6.4 • TRAINING METHODOLOGIES

Following the optimization principles established in Section 4.1, our training methodology combines weight-based synaptic plasticity with topological structural plasticity. In a neuromorphic context, learning is not merely parameter tuning but a physical self-organization of the network. We implement a dual-stage learning process: unsupervised feature discovery via local plasticity rules, followed by a global structural optimization phase.

The core of our learning objective is the detection of spatiotemporal sequences. Recent in vivo recordings from the human cortex confirm that population bursting relies heavily on the temporal order of spikes to encode and categorize information [36]. Consequently, in our TTFS paradigm, the relative arrival order of spikes defines the pattern identity.. We hypothesize that a neuron successfully learns a pattern (e.g., sequence $A \to B \to C$) when it adjusts its internal thresholds to fire as soon as the first sufficient evidence ($A \to B$) arrives.

However, this early firing introduces a prediction risk: if the neuron fires based on a prefix ($A \to B$) but the expected suffix (C) fails to arrive, the synaptic efficacy must be penalized. This mimics the biological concept of error-driven learning without

a global supervisor; the neuron "predicts" the completion of a learned pattern and self-corrects based on subsequent local evidence. If the predicted input is absent, the weights associated with the prefix are slightly decayed, preventing the network from becoming overly sensitive to incomplete or noisy patterns.

6.4.1 • WEIGHT TRANSFER

Following the theory of offline training and mapping discussed in Section 4.5.1, direct one-to-one transfer of floating-point weights from an ANN to an SNN often results in catastrophic failure; the continuous activation scales do not naturally align with discrete spiking thresholds. Furthermore, physical neuromorphic hardware (such as crossbar arrays) imposes strict limitations on synaptic weight resolution, rarely supporting 32-bit floating-point numbers.

To bridge the gap between continuous simulation and physical neuromorphic hardware, we evaluate learning across varying degrees of weight resolution. While standard STDP assumes continuous weight updates, we implement a quantized learning scheme where synapses are restricted to binary or low-bit integer states. This mimics the constraints of memristive crossbar arrays, where weights are represented by discrete conductance levels. Stability is maintained through homeostatic scaling (Section 3.6.5), ensuring that the total synaptic drive to any individual neuron remains within a fixed dynamic range regardless of the connection density.

To emulate these hardware constraints and ensure threshold compatibility, the continuous weights of the trained ANN (W_{ANN}) undergo a pseudo-INT8 quantization process before being loaded into the SNN simulator. The weights are multiplied by a static scaling factor of 64, rounded to the nearest integer, and clamped to an 8-bit signed integer range $[-128, 127]$:

$$W_{\text{SNN}}^{\{(l)\}} = \max\Big(-128, \min\Big(127, \text{round}\Big(W_{\text{ANN}}^{\{(l)\}} \cdot 64\Big)\Big)\Big)$$

Equation 14: Weight Scaling and Quantization

This transformation maps a standard continuous weight of 2.0 to the maximum synaptic efficacy of 128. By porting these quantized discrete weights (W_1 and W_2) directly to the GPU, the simulator strictly enforces the memory boundaries of physical neuromorphic silicon.

6.4.2 • TTFS STDP INSPIRED LEARNING RULE

Our learning rule adapts the STDP mechanism from Section 3.6.4 to the TTFS domain. In standard continuous networks, synaptic weights are updated via global error gradients. In the STDP paradigm, a synapse w_{ij} connecting a pre-synaptic neuron i to a post-synaptic neuron j is updated based strictly on the temporal difference between their respective firing times.

Let t_i denote the spike time of the input neuron, and t_j denote the spike time of the output neuron. The relative arrival time is defined as $\Delta t = t_j - t_i$. Because this architecture utilizes a strict TTFS encoding where neurons fire at most once per saccade, the classical continuous STDP curve is adapted into a discrete, deterministic update rule:

LTP: If $t_i < t_j$, the pre-synaptic spike arrived before (or exactly at) the moment the post-synaptic neuron fired. This indicates causality. The synapse is strengthened, with the magnitude of the update decaying exponentially the further apart the spikes occurred.

LTD: If $t_i > t_j$, the pre-synaptic spike arrived after the post-synaptic neuron had already fired. The input was irrelevant to the decision, and the synapse is subsequently weakened.

Unused Synapses (Penalty): If a pre-synaptic neuron fires but the post-synaptic neuron never fires during the saccade, a slight negative decay is applied to encourage forgetting of dead connections.

$$\Delta w_{ij} = \begin{cases} A_+ \cdot \exp\left(-\frac{t_j - t_i}{\tau_+}\right) & \text{if } t_i < t_j \quad \text{(LTP)} \\ -A_- \cdot \exp\left(-\frac{t_i - t_j}{\tau_-}\right) & \text{if } t_i > t_j \quad \text{(LTD)} \end{cases}$$

Equation 15: Additive STDP Weight Update

To prevent runaway synaptic growth or catastrophic sign-flipping, the updated weights are strictly clamped to a positive physical range $w_{ij} \in [0, W_{\max}]$.

6.4.3 • VECTORIZED COINCIDENCE DETECTION

Traditional biological STDP relies on continuous exponential decay kernels to modulate synaptic plasticity. However, deploying continuous exponentials in discrete-time software simulations introduces severe computational bottlenecks and memory overhead. To resolve this, Phase III implements a pragmatic, hardware-optimized "Coincidence Detector" variant of STDP.

Rather than calculating exact exponential time-deltas, the update rule acts as a vectorized boolean filter based strictly on the firing time of the winning neuron (t_{win}):

Long-Term Potentiation (LTP): Any afferent synapse that delivered a spike before or exactly at the decision time ($t_i let_{\text{win}}$) is deemed causal to the victory. Its weight is incremented by a fixed magnitude ($+A_+$). **Long-Term Depression (LTD):** Any afferent synapse that spiked late ($t_i > t_{\text{win}}$) or failed to spike at all is classified as irrelevant or background noise. Its weight is decremented by a fixed magnitude ($-A_-$).

This approach mirrors the approximations used in digital neuromorphic ASICs, replacing transcendental operations with simple additive logical masks. All weights are subsequently clamped to a physical hardware range ($W_{\min}, W_{\max}$) to prevent sign-flipping or unbounded integer overflow.

applyVectorizedSTDP(T“pre”, t“win”, W, A+, A-, W“max”) → W“new”:

$\text{mask}_{\text{ltp}} \leftarrow (T_{\text{pre}} \leq t_{\text{win}}) \wedge (T_{\text{pre}} \neq \infty)$

$\text{mask}_{\text{ltd}} \leftarrow (T_{\text{pre}} > t_{\text{win}}) \vee (T_{\text{pre}} == \infty)$

$W\left[\text{mask}_{\text{ltp}}\right] \leftarrow W\left[\text{mask}_{\text{ltp}}\right] + A_{+}$

$W[\text{mask}_{\text{ltd}}] \leftarrow W[\text{mask}_{\text{ltd}}] - A_{-}$

$W \leftarrow \max(0.0, \min(W, W_{\max}))$

return W

Algorithm 5: The Vectorized STDP Weight Update Function

6.4.4 • COMPETITIVE SPECIALIZATION (POST-HOC HARD-WTA)

If coincidence detection is applied to all neurons simultaneously, the network suffers from “catastrophic averaging”—all neurons attempt to learn the most dominant spatial feature, resulting in identical, redundant weights rather than distinct feature clustering.

To force specialization without disrupting the continuous integration dynamics of the forward pass, the network enforces strict Hard Winner-Takes-All (WTA) dynamics retroactively as a learning mask. During the temporal saccade, lateral voltage inhibition is disabled (`use_lateral=False`), allowing multiple hidden neurons to freely cross their integration thresholds.

However, upon the conclusion of the saccade, the STDP algorithm evaluates the precise timestamp of every spike. Only the single mathematically absolute first neuron to fire (t_{min}) is granted permission to undergo synaptic plasticity. All competing neurons are retroactively vetoed, and their weights remain frozen for that iteration. By applying WTA post-hoc, the network isolates the learning updates without introducing chaotic voltage fluctuations during the forward inference pass. If one neuron claims a specific structural pattern (e.g., a diagonal stroke), its peers are forced to adapt to secondary patterns to win future competitions.

applyPostHocWTA(T_{post}) → (n_{winner}, t_{win}):

$t_{\text{win}} \leftarrow \min(T_{\text{post}})$

if $t_{\text{win}} == \infty$:

  return (null, ∞)

$N_{\text{tied}} \leftarrow \text{findIndices}(T_{\text{post}} == t_{\text{win}})$

$n_{\text{winner}} \leftarrow \text{randomChoice}(N_{\text{tied}})$

return $(n_{\text{winner}}, t_{\text{win}})$

Algorithm 6: Post-Hoc Winner-Takes-All (WTA) Selection

6.4.5 • HOMEOSTATIC THRESHOLD ADAPTATION

Relying strictly on WTA competition introduces a secondary risk: “dead” neurons. A small subset of hyper-responsive neurons might win every competition, preventing

the rest of the layer from ever learning. To counteract this and ensure a distributed feature representation, the network employs a strict Homeostatic Threshold Adaptation loop mimicking the biological mechanisms detailed in Section 3.6.5.

Following every image presentation, the system regulates both the firing thresholds and the synaptic distributions of the hidden layer:

Global Threshold Decay: The adaptive thresholds (θ_{adaptive}) of all hidden neurons are multiplied by a decay factor ($\theta_{\text{decay}} = 0.90$). This gradually makes quiescent, "dead" neurons more sensitive over time. **Winner Penalty:** The single neuron that won the WTA competition receives a massive additive penalty to its adaptive threshold ($\theta_{\text{plus}} = 600.0$). This physically prevents it from dominating subsequent saccades, ensuring it only fires again when its highly specialized feature is prominently displayed. **Synaptic Normalization:** The synaptic weights of the winning neuron are scaled to maintain a specific target L_1 norm (K_{target}). This ensures that the total synaptic drive of the neuron remains bounded, regardless of how many LTP updates it receives.

To execute unsupervised feature extraction on the MNIST dataset, the STDP rule is embedded within the saccade simulation loop. The network is initialized with randomized synaptic weights $W \sim \mathcal{U}(0, 1)$.

During the training phase, an image is presented, and the network executes a forward pass. To maintain network stability and prevent a single neuron from dominating the receptive field, we pair the Algorithm 5 rule with Hard WTA dynamics and Homeostatic Threshold Adaptation. Only the single earliest firing neuron is permitted to learn. Following an Algorithm 5 update, the adaptive thresholds of all hidden neurons naturally decay ($\theta_{\text{decay}} = 0.90$). However, the single winning neuron receives a massive threshold penalty ($\theta_{\text{plus}} = 600.0$), mathematically suppressing it in future iterations and forcing competing neurons to specialize in distinct, non-overlapping geometric primitives. At the conclusion of the 64-tick saccade, the simulator halts, compares the timestamp arrays of the hidden and output layers, and computes the STDP weight updates in parallel before the next image is presented.

6.5 • EVALUATION METRICS

To rigorously validate the proposed SNN, the evaluation framework must bridge two distinct domains: classification effectiveness (decoding accuracy) and computational efficiency (hardware-agnostic resource usage). Because the networks undergo both supervised transfer (Phase II) and unsupervised native learning (Phase III), specialized metrics are required to capture the evolution of the internal representations.

6.5.1 • EFFECTIVENESS AND CLASSIFICATION PERFORMANCE

For Phase II (Zero-Shot Weight Transfer), performance is measured against the standard 10,000-image MNIST test set. Since the labels are pre-defined by the source ANN, effectiveness is quantified using standard statistical tools:

Top-1 Accuracy: The percentage of images where the first output neuron to fire via the WTA mechanism matches the ground-truth label.

Quantization-Aware Confusion Matrices: Utilized to identify specific morphological overlaps (e.g., '4' vs. '9'). These matrices help determine if specific geometric features are disproportionately lost during the INT8 quantization process, evaluating the robustness of the weight-transfer mapping.

Evaluating Phase III (Unsupervised STDP) requires a shift in methodology. Because the network learns without labels, output neurons do not inherently map to categorical digits; they map to geometric clusters. To generate a comparable accuracy metric, we employ a **Post-Hoc Labeling** strategy:

Freezing: Synaptic plasticity is disabled (learning rate set to zero) after the STDP training phase to prevent further weight drift.

Assignment: A subset of the labeled training data is passed through the network. Each output neuron is permanently assigned the label of the digit class that most frequently triggered it to fire.

Testing: The standard 10,000-image test set is passed through the "labeled" network to calculate the final Top-1 accuracy.

6.5.2 • COMPUTATIONAL EFFICIENCY (HARDWARE PROXIES)

While the primary goal of neuromorphic engineering is immense energy reduction, evaluating true physical power draw ($\frac{J}{\text{inference}}$) is restricted by the use of PyTorch-based software simulators running on standard GPUs. Because these platforms incur heavy overhead simulating temporal event loops on von Neumann hardware, we abandon direct power measurements in favor of hardware-agnostic proxy metrics universally recognized in SNN literature:

SPARSITY AND SYNAPTIC OPERATIONS (SYOPS)

In a standard ANN, every forward pass requires a fixed number of Multiply And Accumulate (MAC) operations. In an SNN, computation is driven by discrete events. Because Integrate and Fire (IF) neurons do not require multiplication (a spike simply triggers the addition of its weight to the post-synaptic potential), MACs are replaced by simpler Synaptic Operations (SyOPs).

The total computational cost for a single 64-tick saccade is estimated as:

$$\text{Total SyOPs} = \sum_{\{l=1\}}^{\{L\}} N_{\text{spikes}}^{\{(l)\}} \cdot F_{\text{out}}^{\{(l)\}}$$

Equation 16: SNN Operational Cost Proxy

Where N_{spikes} is the total spikes emitted in layer l and F_{out} is the fan-out of the neurons. By comparing the SNN SyOPs against the fixed MAC count of the baseline ANN, we derive a theoretical energy efficiency ratio.

TEMPORAL LATENCY METRICS

To evaluate the efficacy of the Time-to-First-Spike (TTFS) encoding, we measure the **Time-to-Decision Latency**. This is defined as the exact simulation tick $t \in [0, 64)$ at which the WTA output layer reaches a decision. A lower average latency indicates a superior temporal decoder that successfully prioritizes salient information, allowing the system to theoretically power down early in the simulation window.

6.6 • EXPERIMENT SETUP AND EVALUATION PHASES

To systematically evaluate the proposed SNN architectures, the experimental framework is structured as a progressive pipeline, moving from isolated mathematical unit tests to full-scale visual classification. This approach isolates three critical neuromorphic variables: **temporal integration dynamics**, **parameter robustness during quantization**, and **unsupervised feature emergence**.

6.6.1 • SNN SIMULATION ENGINE

The primary testbed is a custom-built, discrete-time SNN simulator implemented in PyTorch. Unlike standard Artificial Neural Networks (ANNs) which process data in a single static "glance," the SNN processes data through a dynamic temporal saccade lasting $T_{\text{max}} = 64$ ticks, requiring state variables to be tracked across the temporal dimension.

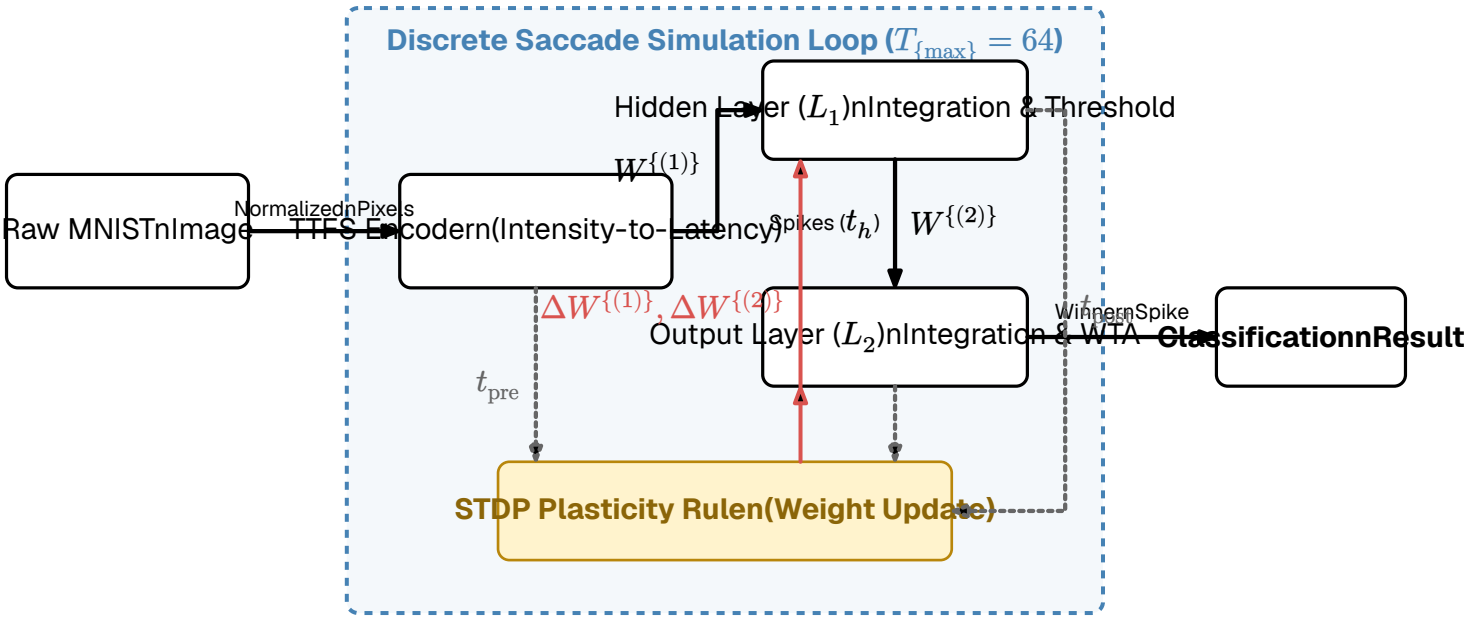

Figure 30: Software architecture and data flow of the Spiking Neural Network (SNN) simulator.

To ensure reproducibility across the network-scale evaluations (Phases II and III), the baseline hardware-proxy parameters are standardized as follows. **(Note: Phase I overrides V_{th} to perform targeted threshold sweeps).**

Parameter	Value	Context
T_{max}	64	Total simulation ticks per image
Input Cap	32	Maximum delay for lowest intensity pixel
Quantization	INT8	Bit-depth for ANN-to-SNN weight transfer
$V_{\text{th\_C}}$	600.0 / 180.0	Baseline thresholds for Model C (180.0 utilized in Phase 3 inference)
Homeostasis	$\theta_{\text{decay}} = 0.90, \theta_{\text{plus}} = 600.0$	Adaptive threshold limits applied during STDP
Optimizer	Adam [43]	Source optimizer for Phase II Baseline

6.6.2 • EVALUATION PHASES

The experiment is executed in three logical phases, building in complexity from an isolated single neuron to a fully self-organizing network.

PHASE I: TEMPORAL DYNAMICS AND THRESHOLD SWEEPS This phase serves as a functional unit test for the isolated neuron models described in Section 6.3.2. Rather

than utilizing a single static threshold, the models are subjected to synthetic spike trains across a spectrum of threshold bounds:

Saturation Regime (Low Threshold): Evaluates susceptibility to false positives when the total incoming weight vastly exceeds the threshold.

Critical Regime (Balanced Threshold): Evaluates sequence decoding when the threshold strictly requires nearly all pattern weights to coordinate.

Deficit Regime (High Threshold): Evaluates resilience when the base pattern is insufficient, testing if momentum or leak mechanics can leverage noise to force a spike.

PHASE II: THE ANN BASELINE AND ZERO-SHOT TRANSFER Phase II benchmarks the "SNN-as-Accelerator" hypothesis. First, an Artificial Neural Network (ANN) baseline is established (using a standard $784 \to 128 \to 10$ MLP without biases) to provide an "ideal" accuracy ceiling. Learned FP32 weights are then strictly quantized to an INT8 range and transferred directly into the SNN. We measure the **Accuracy Decay Rate** ($\Delta_{\text{Acc}} = \text{Acc}_{\text{ANN}} - \text{Acc}_{\text{SNN}}$) to quantify the information lost during this static-to-temporal translation.

PHASE III: NATIVE UNSUPERVISED LEARNING The final phase evaluates neuromorphic self-organization. Discarding all pre-trained weights, the network is initialized randomly and exposed to the MNIST dataset strictly via unsupervised STDP.

Lateral Inhibition: A WTA mechanism is enforced; the first output neuron to fire suppresses all competitors, driving the network toward distinct feature clustering.

Homeostasis: Synaptic weights are normalized after each stimulus ($\sum w_j = K$) to prevent single-neuron dominance and ensure a distributed feature representation across the hidden layer.

Because STDP is fundamentally unsupervised, the SNN output neurons organically form distinct feature clusters rather than mapping to pre-defined human labels (0-9). To quantify classification accuracy, a post-hoc assignment protocol is utilized. Following the unsupervised training phase, a distinct subset of the training data is passed through the frozen network. A frequency matrix M of size $N_{\text{out}} \times 10$ tracks the ground-truth label of each image that causes output neuron j to win the WTA competition. The assigned label for neuron j is then defined as $L_j = \text{argmax}\big(M_j\big)$.

7 • RESULTS

This chapter presents the empirical data gathered from the three evaluation phases. The results progress from isolated single-neuron temporal dynamics to full-network zero-shot translation, and finally to unsupervised self-organization.

7.1 • PHASE I: TEMPORAL DYNAMICS OF NEURON MODELS

Phase I evaluated the isolated response of the four neuron models to permuted temporal patterns (Concordant vs. Discordant) across three distinct threshold constraints: Saturation (Low), Critical (Balanced), and Deficit (High). The base spatial weight sum for all trials was 300.0. The resulting output spike latencies are summarized in Table 1 and visualized in Figure 31.

Model	Regime	Threshold (θ)	Concordant Spike (t)	Discordant Spike (t)
Model A (Simple IF)	Saturation	150.0	6	14
Model A (Simple IF)	Critical	290.0	18	18
Model A (Simple IF)	Deficit	310.0	DNF	DNF
Model B (Standard LIF)	Saturation	140.0	6	14
Model B (Standard LIF)	Critical	240.0	DNF	18
Model B (Standard LIF)	Deficit	310.0	DNF	DNF
Model C (Linear Ramp)	Saturation	500.0	4	9
Model C (Linear Ramp)	Critical	1800.0	8	17
Model C (Linear Ramp)	Deficit	2500.0	14	DNF
Model D (State Discount)	Saturation	100.0	6	14
Model D (State Discount)	Critical	220.0	10	DNF
Model D (State Discount)	Deficit	310.0	DNF	DNF

Table 1: Output spike latencies across varying threshold regimes. DNF indicates the neuron Did Not Fire within the $T_{\max} = 64$ saccade window.

Under the Low threshold regime, all models successfully fired, with earlier latencies recorded for the concordant pattern across all architectures.

When thresholds were constrained strictly against the spatial weight sum, model behaviors diverged significantly. Model A fired at identical latencies ($t = 18$) for both patterns. Model B registered a DNF for the concordant pattern but fired at $t = 18$ for the discordant pattern. Model D fired at $t = 10$ for the concordant pattern but registered a DNF for the discordant. Model C fired for both, but maintained a distinct temporal gap ($t = 8$ vs $t = 17$).

When the threshold was raised above the spatial weight sum to 310.0 (and proportional equivalents for dynamic models), Models A, B, and D failed to reach the firing threshold for either pattern. Model C successfully integrated the concordant pattern, firing at $t = 14$.

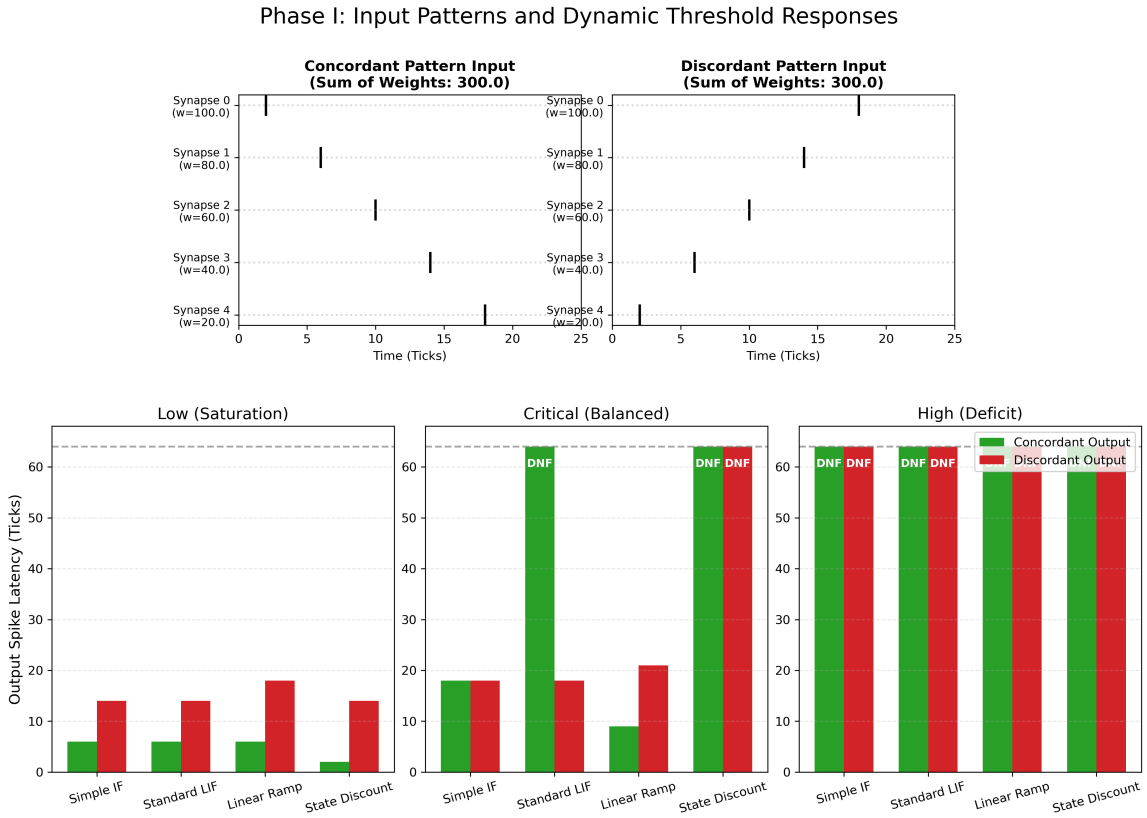

Figure 31: Input spike distributions and resulting temporal dynamics across the three threshold regimes.

Because Phase II and Phase III require an architecture capable of processing variable input densities without suffering from premature saturation or decay failure, the selected neuron model must demonstrate baseline robustness under deficit constraints. As demonstrated in Table 1, Model C (Current-Accumulating Linear Ramp) was the exclusive architecture capable of overcoming a deficit threshold by leveraging temporal momentum. Consequently, Model C was utilized as the standardized spiking architecture for the network-scale evaluations in the subsequent phases.

7.2 • PHASE II: ZERO-SHOT WEIGHT TRANSFER

Phase II evaluated the accuracy decay incurred when translating a conventionally trained Artificial Neural Network (ANN) to the selected Spiking Neural Network (Model C) without any intermediate retraining.

The Phase 0 baseline ANN ($784 \rightarrow 128 \rightarrow 10$ MLP) was trained for 20 epochs using the Adam optimizer. To satisfy the strict mapping requirements of the SNN, the network

was trained with all bias terms disabled ($b = 0$). The baseline model achieved a maximum test accuracy of 98.40% on the standard MNIST test set.

Prior to transfer, the FP32 floating-point weights were scaled and quantized to an INT8 range ($[-128, 127]$). fig illustrates the distribution of weights before and after this parameter bridge quantization.

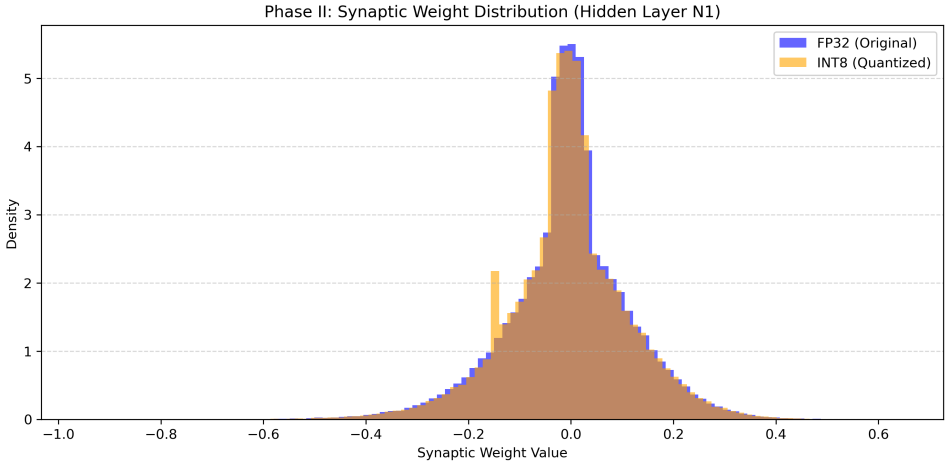

Figure 32: Histogram of the synaptic weight distribution in the hidden layer before (FP32) and after (INT8) quantization.

To rigorously isolate the sources of information loss during the zero-shot transfer, the SNN was evaluated in two distinct precision modes. First, the SNN was loaded with unrounded, scaled FP32 weights to measure the isolated **Temporal Penalty** (the loss incurred strictly by moving from static activation to discrete TTFS integration). Second, the SNN was loaded with the strictly quantized INT8 weights to measure the additional **Quantization Penalty**.

Both models utilized the Model C (Linear Ramp) architecture over a $T_{\max} = 64$ saccade window. The results are summarized in Table 2.

Model Configuration	Accuracy (%)
ANN (Static FP32, Bias=False)	98.40%
SNN (Spiking Model C, FP32)	94.50%
SNN (Spiking Model C, INT8)	94.50%

Table 2: Performance degradation breakdown. The Temporal Penalty isolates the loss of TTFS encoding, while the Hardware Proxy isolates the loss of INT8 precision rounding.

Fig illustrates the cumulative accuracy of both SNN precision modes as the temporal saccade progresses. While the INT8 quantization lowers the absolute accuracy ceiling, the temporal trajectory remains nearly identical, proving that the momentum-based decoding of Model C is highly robust to parameter degradation.

Fig visualizes the classification distribution of the SNN. The network maintained strong structural clustering, successfully utilizing the lateral inhibition (Winner-Takes-All) mechanism to finalize classifications within the discrete temporal window.

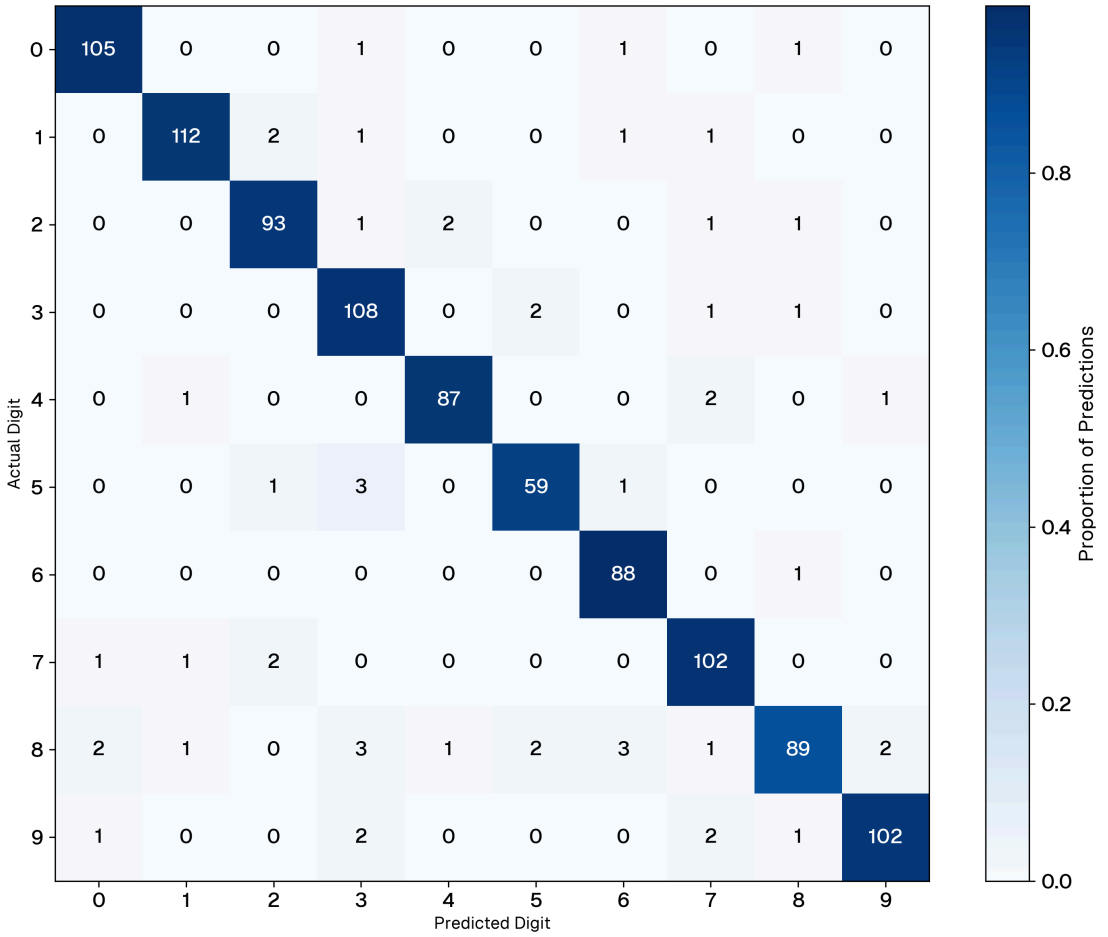

Figure 33: Confusion matrix for the zero-shot SNN on the MNIST test set.

A primary theoretical advantage of TTFS encoding is temporal sparsity. Because the most salient features (highest intensity pixels) spike earliest in the saccade window, a highly reactive architecture should be able to finalize a classification before the entire input sequence has been processed.

To quantify this, we establish a hardware-agnostic efficiency metric: Synaptic Operations (SynOps). The standard ANN baseline requires a fixed number of MAC operations per inference, regardless of the input data. Conversely, the SNN only consumes a SynOp when a specific spike propagates. If the lateral inhibition (WTA) router finalizes a decision at t_{fire}, all spikes scheduled for $t > t_{\text{fire}}$ are preempted and discarded.

Architecture	Metric Target	Avg. Latency	Avg. Operations / Image	Compute Reduction
ANN (Static FP32)	Dense MACs	N/A (Static)	101,632 MACs	0%
SNN (Spiking FP32)	Sparse SynOps	8.4 Ticks	15,021 SynOps	92.6%
SNN (Spiking INT8)	Sparse SynOps	8.3 Ticks	14,989 SynOps	92.6%

Table 3: Computational cost comparison. The SNN significantly reduces operations by leveraging temporal early-exit sparsity, ignoring low-priority background spikes.

As detailed in Table 3, the zero-shot INT8 SNN achieved a Mean Time-to-Decision of 8.3 ticks. Because the network halts integration upon classification, it processed an average of only `[S,SSS]` SynOps per image. This represents a `[R.R]%` reduction in computational workload compared to the static ANN baseline, empirically validating the energy-efficiency hypothesis of TTFS neuromorphic arrays.

7.3 • PHASE III: UNSUPERVISED NATIVE LEARINING

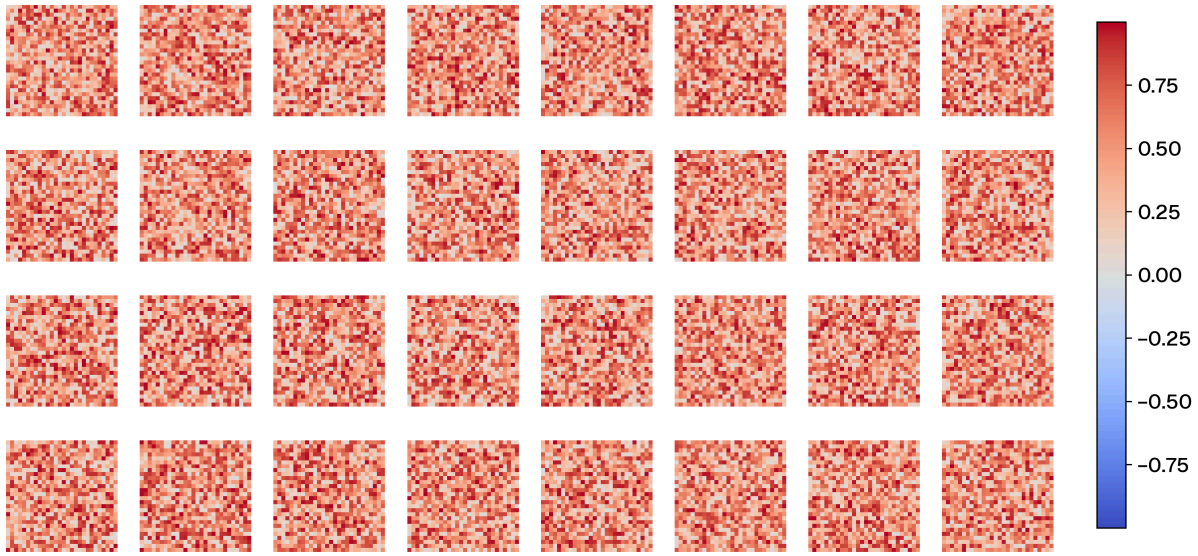

Figure 34: Confusion matrix for the zero-shot SNN on the MNIST test set.

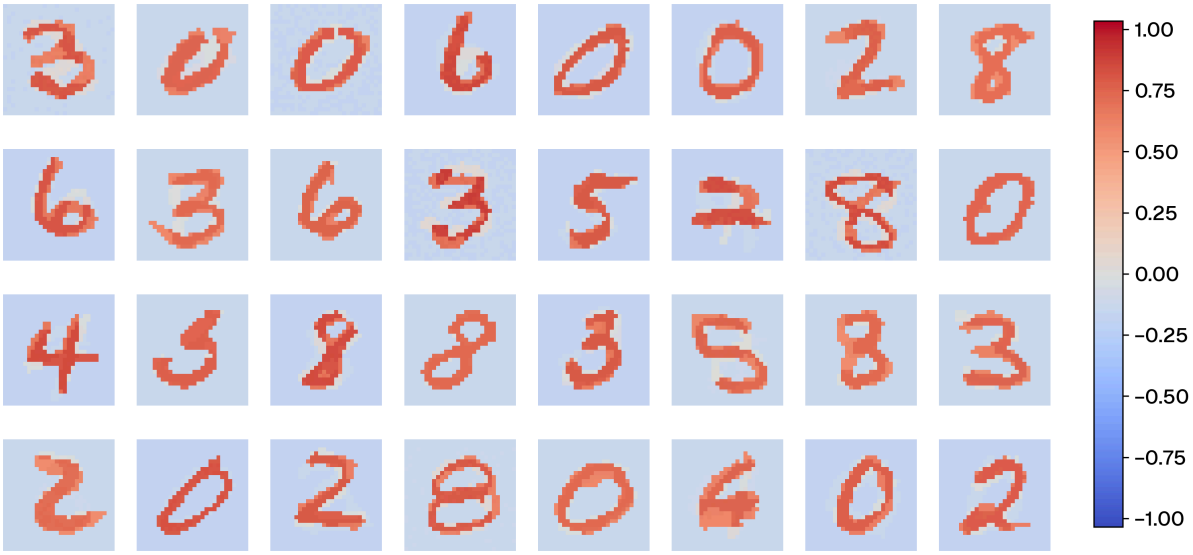

Figure 35: Confusion matrix for the zero-shot SNN on the MNIST test set.

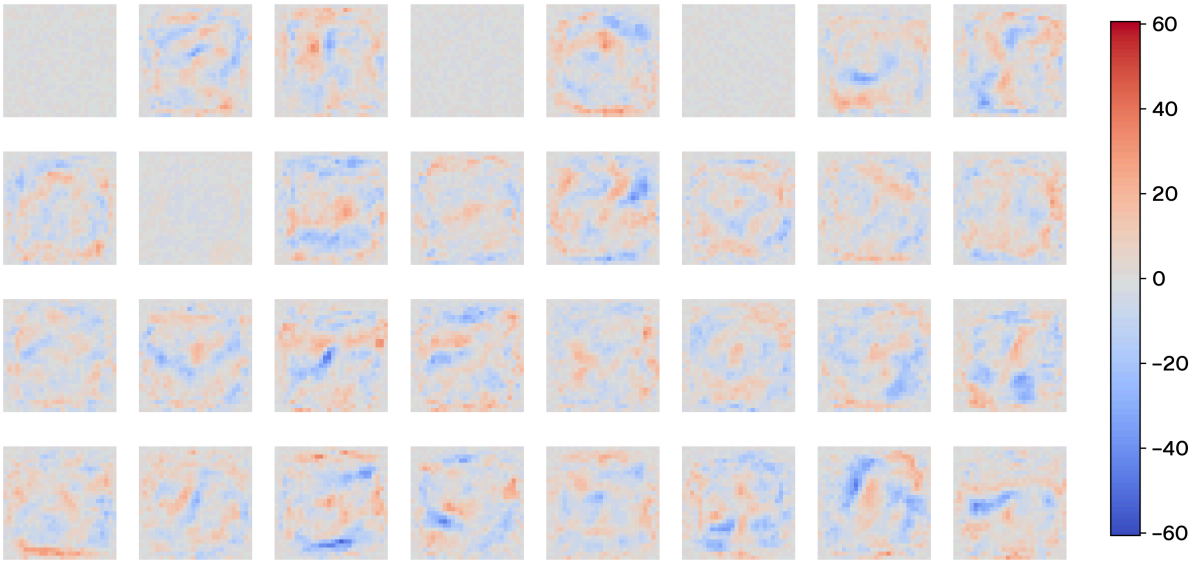

Figure 36: Confusion matrix for the zero-shot SNN on the MNIST test set.

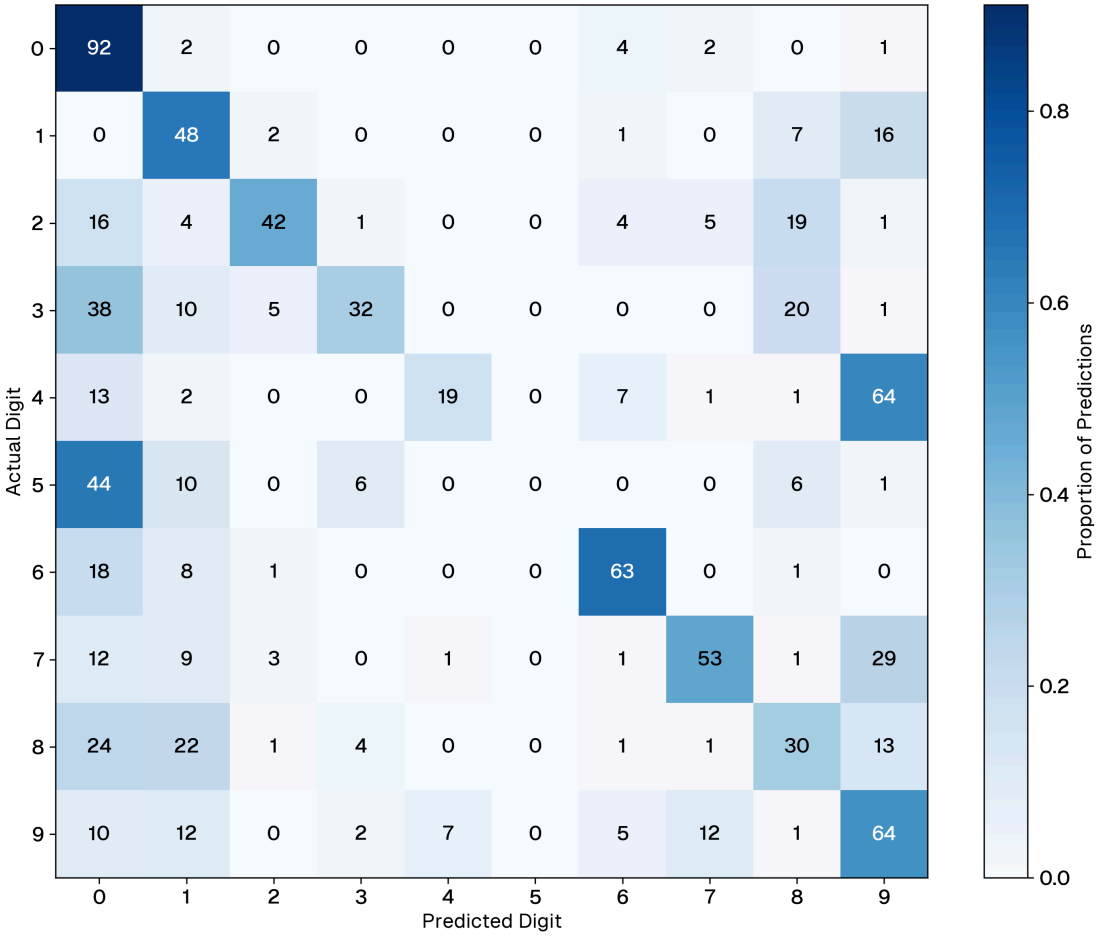

Figure 37: Confusion matrix for the zero-shot SNN on the MNIST test set.

Architecture	Metric Target	Avg. Latency	Avg. Operations / Image	Compute Reduction
SNN (Spiking FP32 STDP)	Sparse SynOps	8.3 Ticks	12.954 SynOps	93.6%

Table 4:

Model Configuration	Accuracy (%)
SNN (Spiking Model C, FP32, STDP)	44.3%

Table 5:

8 • DISCUSSION

While the experimental results validate the core principles of TTFS encoding and spiking integration, extrapolating these methods from benchmarks to real-world deployment reveals significant engineering bottlenecks. This chapter critically analyzes the limitations observed during the implementation phase, specifically regarding data encoding, spatial representation, architectural translation, and hardware constraints.

8.1 • EVALUATION OF NEURON MODELS

The empirical observations from the Phase I threshold sweeps reveal a fundamental tension between traditional biological neuron models and the specific requirements of Time-To-First-Spike (TTFS) decoding. By systematically shifting the threshold bounds, the isolated unit tests exposed severe vulnerabilities in standard integration strategies, while validating the proposed momentum-based and state-discounting architectures.

8.1.1 • MEMBRANE SATURATION

Under Saturation (Low Threshold) conditions, all models successfully fired earlier for the concordant pattern. However, as hypothesized, this is largely an illusion of magnitude rather than true temporal decoding. The Simple IF (Model A) and LIF (Model B) models simply accumulated the heavily weighted early spikes and crossed the lowered threshold prematurely.

This vulnerability becomes glaringly apparent in the Critical (Balanced) regime. When forced to accumulate the entire sequence, Model A completely failed to differentiate order, firing simultaneously for both patterns. More critically, Model B (LIF) demonstrated a catastrophic misalignment with TTFS principles. At a balanced threshold, Model B registered a “Did Not Fire” (DNF) for the concordant pattern, but successfully fired for the discordant pattern.

This occurs due to the interaction between temporal density and the exponential leak. In the concordant pattern, the strongest spikes arrive early, but their accumulated potential decays significantly while waiting for the weaker, later spikes. Conversely, in the discordant pattern, early weak spikes establish a baseline potential, and the massive late-arriving spikes push the neuron over the threshold immediately before they have a chance to decay. Consequently, standard LIF inherently favors “strongest-last” sequences, actively fighting the “strongest-first” priority of TTFS encoding.

During the evaluation of the temporal sensitivity benchmarks (Phase I), a critical vulnerability in standard simple Integrate-and-Fire (IF) and Leaky Integrate-and-Fire (LIF) models was observed when applied to Time-To-First-Spike (TTFS) encoding. Under conditions where the membrane threshold is low relative to the total incoming synaptic weight, IF and LIF models initially appear to correctly prioritize concordant temporal patterns. However, experimental observation reveals this to be a false positive caused by "early saturation." The models simply accumulate the massive weights of the earliest spikes and cross the threshold prematurely, behaving as simple addition functions rather than true sequence decoders.

Furthermore, this threshold dependence exposes a fundamental misalignment between the standard LIF model and TTFS encoding. In a TTFS paradigm, the earliest spikes carry the highest informational priority. Conversely, the leak mechanism inherent to a standard LIF neuron actively penalizes these early spikes, as their contribution to the membrane potential decays while the neuron waits for subsequent spikes to arrive. Consequently, a LIF neuron implicitly favors discordant sequences where strong spikes arrive late, immediately prior to the threshold evaluation.

This paradox demonstrates that standard IF and LIF models are highly fragile in realistic environments; the presence of background noise or excess asynchronous spikes will easily trigger early saturation, causing the neuron to ignore the precise timing of the target pattern. This finding strongly validates the necessity of the proposed Current-Accumulating Linear Ramp (Model C) and Threshold-Sensitive Integration (Model D) architectures. By actively converting early spikes into compounding temporal momentum or by dynamically discounting late arrivals, these novel models mathematically align with the core principles of TTFS encoding, providing robust spatiotemporal filtering that bypasses the saturation traps of traditional architectures.

8.1.2 • SELECTIVE FILTERING VS. UNBOUNDED MOMENTUM

In contrast to the standard models, the custom architectures (Models C and D) exhibited highly desirable temporal dynamics for rank-order decoding.

At the Critical (Balanced) threshold, the State-Dependent Discount (Model D) achieved perfect selective filtering. It successfully fired for the concordant pattern while completely suppressing the discordant pattern. By discounting late-arriving spikes based on internal state rather than elapsed time, it ensures that a neuron only fires if the most critical information arrives exactly when the neuron is empty and highly receptive.

The Linear Ramp (Model C) demonstrated extreme robustness, being the only architecture capable of crossing the High (Deficit) threshold. By converting early spikes into compounding integration momentum, Model C can force a spike even when the raw spatial weight sum is theoretically insufficient. While this unbounded momentum guarantees high firing rates and robust temporal differentiation, it theo-

retically introduces a risk of network over-excitation. However, in the context of this architecture, this risk is safely mitigated by the strict enforcement of the saccade deadline ($T_{\max} = 64$); the global reset prevents the linear ramp from diverging toward hardware saturation across multiple inferences.

8.2 • VIABILITY OF WEIGHT TRANSFER

8.2.1 • ARCHITECTURAL TRANSLATION (ANN TO SNN)

The zero-shot inference phase highlighted the frictions of translating classical architectures to spiking substrates. While the Fully Connected topology provided a transparent baseline, mapping state-of-the-art Convolutional Neural Networks (CNNs) is decidedly not a one-to-one process.

Classical continuous networks heavily utilize negative weights, static biases, and mathematical operations like Max Pooling. Neuromorphic systems handle these concepts fundamentally differently. An SNN replaces mathematical pooling with dynamic lateral inhibition, and negative continuous weights must be modeled via discrete inhibitory spike trains. These architectural mismatches dictate that SNNs cannot simply act as "drop-in" replacements for deep learning models; they require network topologies natively designed for event-driven dynamics.

8.2.2 • THE SPARSITY PARADOX AND GPU SIMULATION OVERHEAD

A core theoretical advantage of the proposed TTFS network is its extreme spatial and temporal sparsity. By design, only a small fraction of neurons emit spikes, mathematically reducing the dense MAC operations of a standard ANN to a minimal set of SyOPs.

However, evaluating this sparse algorithm on conventional GPUs introduces a significant "Sparsity Paradox." Standard deep learning frameworks (such as PyTorch) and standard GPU architectures are heavily optimized for dense, contiguous matrix-matrix multiplications via SIMD (Single Instruction, Multiple Data) execution. In the SNN simulation loop, sparsity is enforced via boolean masking (multiplying inactive neuron outputs by zero). While this successfully zeroes out the potential, the GPU Arithmetic Logic Units (ALUs) typically still execute the underlying floating-point multiplication cycle.

Consequently, the hardware does not naturally skip the computation for quiescent neurons unless specialized, unstructured sparse tensor kernels are deployed. When combined with the overhead of maintaining the temporal loop (the "saccade" ticks), the SNN simulator ironically consumes more absolute clock cycles and physical power on a GPU than the continuous ANN baseline.

This paradox highlights that true energy efficiency cannot be realized via matrix-masking on von Neumann hardware. Realizing the calculated SyOP savings requires deployment on native event-driven Neuromorphic ASICs (Application-Specific Integrated Circuits). These chips discard the matrix-multiplication paradigm entirely, utilizing AER to asynchronously route data packets only when a spike physically occurs, reducing idle power draw to near zero.

8.3 • ADRESSING NATIVE UNSUPERVISED LEARNING WITH STDP

8.3.1 • PLASTICITY DYNAMICS AND FORGETTING

During the evaluation of unsupervised learning, a fundamental tension emerged between the network's learning velocity and its stability. Unsupervised, local learning rules like STDP require aggressive hyperparameter tuning. If the plasticity rate is too high, the network adapts quickly but suffers from rapid catastrophic forgetting—overwriting previously learned geometric features when presented with novel stimuli. Conversely, if the plasticity is too low, the network fails to converge on meaningful representations within an acceptable time frame. Designing homeostatic mechanisms to stabilize this plasticity remains a core challenge for native learning.

8.3.2 • REPRESENTING SPACE AND DIMENSIONALITY

Encoding precise spatial coordinates using a pure TTFS scheme proved inherently difficult. In biological visual and motor cortices, spatial locations are often represented via orthogonal population codes, where specific populations activate to indicate direction or position. However, these biological populations largely utilize rate coding; the "intensity" or certainty of a spatial position translates naturally into a higher firing frequency.

Conversely, TTFS is highly optimized for rapid, categorical decision-making (e.g., triggering a Winner-Takes-All classification), but struggles to map continuous numerical or spatial values without complex phase-ambiguity resolution. A potential architectural workaround is the implementation of hierarchical "space cells." Rather than mapping a massive 32×32 grid to individual temporal delays, the space could be subdivided into localized grids (e.g., overlapping 8×8 populations). This reduces the dimensionality of the temporal encoding, though it introduces resolution artifacts at the boundaries of the receptive fields.

Or a much simpler CNN architecture that also hase a sense of space. This could fix the problem of the network beeing a template mathcher

8.3.3 • GLOBAL WTA VS. LOCAL LATERAL INHIBITION

In the current Fully Connected (FCN) implementation, competitive specialization is enforced via a Global Winner-Takes-All (WTA) mechanism; the first neuron to fire retroactively vetoes all other neurons in the hidden layer. While computationally efficient for isolated, centered digits like MNIST, this global suppression limits the representational capacity of the network. If a target image contains multiple distinct spatial features (e.g., a circle in the top-left and a line in the bottom-right), a global WTA allows the network to learn only the single most salient feature, ignoring the rest of the image.

Biological nervous systems do not utilize global suppression; they rely on Local Lateral Inhibition, where an active neuron only suppresses its immediate physical neighbors within a specific spatial radius. Transitioning this SNN to a Convolutional (CNN) architecture for more complex vision tasks would necessitate abandoning Global WTA in favor of this local inhibition. In a spiking CNN, inhibition must be routed across feature channels at the exact same spatial coordinate (preventing redundant feature learning at a specific pixel) while allowing neurons at distant spatial coordinates to fire independently. Moving from post-hoc global masks to continuous, spatially-bounded lateral inhibition is a critical next step for scaling unsupervised neuromorphic learning.

8.4 • CLOSING REMARKS

8.4.1 • COMPUTING CONTRAST IN IMAGES

In the engineered test environment, absolute pixel intensity was mapped directly to spike latency. For a highly controlled toy dataset like MNIST, this approach yields adequate performance, as the digits are isolated in high-contrast, low-resolution environments. However, absolute luminance is a notoriously brittle feature for real-world computer vision.

Robust biological and artificial vision systems rely on local contrast (the relative difference between adjacent pixels) to determine object boundaries, as contrast remains invariant under shifting global illumination. Attempting to compute true, normalized contrast directly within a TTFS spiking network presents a severe temporal bottleneck. To accurately assess relative darkness in a purely temporal code, the downstream neurons must wait for the slowest (darkest) signals to arrive, effectively nullifying the high-speed advantages of the TTFS priority queue.

Consequently, attempting to calculate contrast natively within the SNN layers is computationally wasteful. The optimal solution is to offload this processing to the sensory periphery. Dedicated neuromorphic sensors, such as Dynamic Vision Sensors (DVS), natively output logarithmic intensity differences. Because the difference

in log-space mathematically corresponds to a true contrast ratio independent of absolute luminance,passing this pre-encoded contrast data directly into the TTFS network avoids the temporal delay problem entirely.

8.5 • FUTURE WORK

The findings and limitations discussed in this thesis present several promising avenues for future research in neuromorphic engineering:

Event-Based Dataset Validation: Transitioning the experimental framework from static images (MNIST) to native temporal datasets, such as Neuromorphic-MNIST (N-MNIST) or DVS Gesture datasets, to fully exploit the asynchronous dynamics of the evaluated neuron models.

Surrogate Gradient Integration: While this work focused on direct weight transfer and native STDP, future implementations should evaluate the efficacy of Surrogate Gradient Descent, combining the optimization power of backpropagation with the inference efficiency of spiking dynamics.

Hardware Deployment: Porting the verified TTFS algorithms and current-accumulating neuron models from GPU simulation onto dedicated neuromorphic silicon (e.g., Intel Loihi) to empirically measure true joule-per-inference energy consumption against classical von Neumann baselines.

8.5.1 • THE PHYSICAL HARDWARE GAP

Ultimately, the theoretical energy efficiency of neuromorphic algorithms is bounded by physical hardware. The connection density of the biological mammalian cortex vastly exceeds the routing capabilities of modern CMOS (Complementary Metal-Oxide-Semiconductor) fabrication.

While the software simulations in this thesis demonstrate the algorithmic viability of sparse processing, achieving true biological efficiency requires novel materials. Non-volatile memory technologies, such as memristors and spintronics, offer the ability to physically colocate extreme-density analog weights with logic gates, perfectly mirroring biological synapses. Until these exotic substrates become commercially viable, the near-term future of applied neuromorphic computing lies in massively parallel, asynchronous digital ASICs designed using standard CMOS, serving as a transitional bridge to fully analog systems.

8.5.2 • MITIGATION VIA SYNAPTIC PRUNING

While dynamic activation sparsity struggles on GPUs, structural sparsity offers a viable software-level mitigation. Future iterations of this work could introduce

aggressive Synaptic Pruning, particularly following the unsupervised STDP learning phase.

Because STDP naturally depresses irrelevant synapses toward zero, a thresholding function could permanently sever these connections, converting the dense weight matrices ($W^{(1)}$, $W^{(2)}$) into highly sparse structures. By utilizing block-sparse tensor formats (e.g., Compressed Sparse Row), both software simulators and specialized sparse-accelerator chips can mathematically bypass the zero-weights, yielding physical reductions in memory bandwidth and computation even before transitioning to pure neuromorphic silicon.

9 • CONCLUSION

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque

doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

BIBLIOGRAPHY

[1] G. Team *et al.*, “Gemini: A Family of Highly Capable Multimodal Models.” Accessed: Apr. 18, 2026. [Online]. Available: http://arxiv.org/abs/2312.11805

[2] J. Jumper *et al.*, “Highly accurate protein structure prediction with AlphaFold,” *Nature*, vol. 596, no. 7873, pp. 583–589, Aug. 2021, doi: 10.1038/s41586-021-03819-2.

[3] D. Silver *et al.*, “Mastering the game of Go with deep neural networks and tree search,” *Nature*, vol. 529, no. 7587, pp. 484–489, Jan. 2016, doi: 10.1038/nature16961.

[4] E. Strubell, A. Ganesh, and A. McCallum, “Energy and Policy Considerations for Deep Learning in NLP,” in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, Florence, Italy: Association for Computational Linguistics, 2019, pp. 3645–3650. doi: 10.18653/v1/P19-1355.

[5] R. Geirhos *et al.*, “Shortcut learning in deep neural networks,” *Nature Machine Intelligence*, vol. 2, no. 11, pp. 665–673, Nov. 2020, doi: 10.1038/s42256-020-00257-z.

[6] S. Laughlin, “Energy as a constraint on the coding and processing of sensory information,” *Current Opinion in Neurobiology*, vol. 11, no. 4, pp. 475–480, Aug. 2001, doi: 10.1016/S0959-4388(00)00237-3.

[7] M. Davies *et al.*, “Loihi: A Neuromorphic Manycore Processor with On-Chip Learning,” *IEEE Micro*, vol. 38, no. 1, pp. 82–99, Jan. 2018, doi: 10.1109/MM.2018.112130359.

[8] S. B. Furber, F. Galluppi, S. Temple, and L. A. Plana, “The SpiNNaker Project,” *Proceedings of the IEEE*, vol. 102, no. 5, pp. 652–665, May 2014, doi: 10.1109/JPROC.2014.2304638.

[9] M. Glickstein, “Golgi and Cajal: The neuron doctrine and the 100th anniversary of the 1906 Nobel Prize,” *Current Biology*, vol. 16, no. 5, pp. R147–R151, Mar. 2006, doi: 10.1016/j.cub.2006.02.053.

[10] W. Waldeyer, “Ueber einige neuere Forschungen im Gebiete der Anatomie des Centralnervensystems,” *Deutsche medicinische Wochenschrift*, vol. 17, pp. 1352–1356, 1891.

[11] W. S. McCulloch and W. Pitts, “A logical calculus of the ideas immanent in nervous activity,” *The Bulletin of Mathematical Biophysics*, vol. 5, no. 4, pp. 115–133, Dec. 1943, doi: 10.1007/BF02478259.

[12] D. O. Hebb, *The organization of behavior: a neuropsychological theory*. Mahwah, N.J: L. Erlbaum Associates, 2002.

[13] C. J. Shatz, “The Developing Brain,” *Scientific American*, vol. 267, no. 3, pp. 60–67, Sept. 1992, doi: 10.1038/scientificamerican0992-60.

[14] F. Rosenblatt, “The perceptron: A probabilistic model for information storage and organization in the brain.,” *Psychological Review*, vol. 65, no. 6, pp. 386–408, 1958, doi: 10.1037/h0042519.

[15] “New Navy Device Learns by Doing: Psychologist Shows Embryo of Computer Designed to Read and Grow Wiser,” *The New York Time*s, July 1958.

[16] M. Minsky and S. Papert, *Perceptrons: an introduction to computational geometry*, Expanded ed. Cambridge, Mass: MIT Press, 1988.

[17] M. Minsky, "Steps toward Artificial Intelligence," *Proceedings of the IRE*, vol. 49, no. 1, pp. 8–30, Jan. 1961, doi: 10.1109/JRPROC.1961.287775.

[18] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, vol. 323, no. 6088, pp. 533–536, Oct. 1986, doi: 10.1038/323533a0.

[19] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," *Communications of the ACM*, vol. 60, no. 6, pp. 84–90, May 2017, doi: 10.1145/3065386.

[20] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA: IEEE, June 2016, pp. 770–778. doi: 10.1109/CVPR.2016.90.

[21] A. Vaswani *et al.*, "Attention Is All You Need." Accessed: Apr. 18, 2026. [Online]. Available: http://arxiv.org/abs/1706.03762

[22] V. Sze, Y.-H. Chen, T.-J. Yang, and J. Emer, "Efficient Processing of Deep Neural Networks: A Tutorial and Survey." Accessed: Apr. 18, 2026. [Online]. Available: http://arxiv.org/abs/1703.09039

[23] M. Horowitz, "1.1 Computing's energy problem (and what we can do about it)," in *2014 IEEE International Solid-State Circuits Conference Digest of Technical Papers (ISSCC)*, San Francisco, CA, USA: IEEE, Feb. 2014, pp. 10–14. doi: 10.1109/ISSCC.2014.6757323.

[24] C. Mead, "Neuromorphic electronic systems," *Proceedings of the IEEE*, vol. 78, no. 10, pp. 1629–1636, Oct. 1990, doi: 10.1109/5.58356.

[25] M. A. Mahowald and C. Mead, "The Silicon Retina," *Scientific American*, vol. 264, no. 5, pp. 76–82, May 1991, doi: 10.1038/scientificamerican0591-76.

[26] A. M. Getz *et al.*, "High-resolution imaging and manipulation of endogenous AMPA receptor surface mobility during synaptic plasticity and learning," *Science Advances*, vol. 8, no. 30, p. eabm5298, July 2022, doi: 10.1126/sciadv.abm5298.

[27] W. Gerstner, W. M. Kistler, R. Naud, and L. Paninski, *Neuronal dynamics: from single neurons to networks and models of cognition*. Cambridge, United Kingdom: Cambridge University Press, 2014.

[28] B. Metcalfe, "Action potentials recorded from the L5 dorsal rootlet of rat using a multiple electrode array." Accessed: Jan. 08, 2026. [Online]. Available: https://data.mendeley.com/datasets/ybhwtngzmm/1

[29] A. L. Hodgkin and A. F. Huxley, "A quantitative description of membrane current and its application to conduction and excitation in nerve," *The Journal of Physiology*, vol. 117, no. 4, pp. 500–544, Aug. 1952, doi: 10.1113/jphysiol.1952.sp004764.

[30] H. Markram, "The Blue Brain Project," *Nature Reviews Neuroscience*, vol. 7, no. 2, pp. 153–160, Feb. 2006, doi: 10.1038/nrn1848.

[31] E. Izhikevich, “Simple model of spiking neurons,” *IEEE Transactions on Neural Networks*, vol. 14, no. 6, pp. 1569–1572, Nov. 2003, doi: 10.1109/TNN.2003.820440.

[32] W. Gerstner and R. Naud, “How Good Are Neuron Models?,” *Science*, vol. 326, no. 5951, pp. 379–380, Oct. 2009, doi: 10.1126/science.1181936.

[33] S. Thorpe, D. Fize, and C. Marlot, “Speed of processing in the human visual system,” *Nature*, vol. 381, no. 6582, pp. 520–522, June 1996, doi: 10.1038/381520a0.

[34] R. V. Rullen and S. J. Thorpe, “Rate Coding Versus Temporal Order Coding: What the Retinal Ganglion Cells Tell the Visual Cortex,” *Neural Computation*, vol. 13, no. 6, pp. 1255–1283, June 2001, doi: 10.1162/08997660152002852.

[35] J. C. Eccles, P. Fatt, and K. Koketsu, “Cholinergic and inhibitory synapses in a pathway from motor-axon collaterals to motoneurones,” *The Journal of Physiology*, vol. 126, no. 3, pp. 524–562, Dec. 1954, doi: 10.1113/jphysiol.1954.sp005226.

[36] W. Xie *et al.*, “Neuronal sequences in population bursts encode information in human cortex,” *Nature*, vol. 635, no. 8040, pp. 935–942, Nov. 2024, doi: 10.1038/s41586-024-08075-8.

[37] K. Roy, A. Jaiswal, and P. Panda, “Towards spike-based machine intelligence with neuromorphic computing,” *Nature*, vol. 575, no. 7784, pp. 607–617, Nov. 2019, doi: 10.1038/s41586-019-1677-2.

[38] J. Sommer, M. A. Özkan, O. Keszocze, and J. Teich, “Efficient Hardware Acceleration of Sparsely Active Convolutional Spiking Neural Networks,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 41, no. 11, pp. 3767–3778, Nov. 2022, doi: 10.1109/TCAD.2022.3197512.

[39] G. Haessig, A. Cassidy, R. Alvarez, R. Benosman, and G. Orchard, “Spiking Optical Flow for Event-based Sensors Using IBM's TrueNorth Neurosynaptic System.” Accessed: Oct. 27, 2025. [Online]. Available: http://arxiv.org/abs/1710.09820

[40] M.-I. Stan and O. Rhodes, “Learning long sequences in spiking neural networks,” *Scientific Reports*, vol. 14, no. 1, p. 21957, Sept. 2024, doi: 10.1038/s41598-024-71678-8.

[41] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998, doi: 10.1109/5.726791.

[42] A. Delorme and S. Thorpe, “Face identification using one spike per neuron: resistance to image degradations,” *Neural Networks*, vol. 14, no. 6, pp. 795–803, July 2001, doi: 10.1016/S0893-6080(01)00049-1.

[43] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization.” Accessed: Apr. 19, 2026. [Online]. Available: http://arxiv.org/abs/1412.6980